FireSense AI Fire and Smoke Detection Using YOLOv8

Submitted in partial fulfillment of the requirements for the degree of

Master of Science

In

Data Science

by

M Janani

25MDT0227

Under the guidance of

PRAVEEN T

Assistant Professor Grade 2

School of Advanced Sciences

VIT Vellore



VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

October 2026

DECLARATION

I, M Janani, registration number 25MDT0227, declare that the dissertation entitled "FireSense AI Fire and Smoke Detection Using YOLOv8", submitted for the award of the degree of Master of Science in Data Science at VIT, is a record of the project work presented under the guidance of PRAVEEN T, Assistant Professor Grade 2, School of Advanced Sciences.

The public dataset, software, publications and experimental evidence used in this dissertation are identified in the text and references. The reported measurements are drawn from the saved project results. External material has been acknowledged where it is used.

I further declare that this dissertation has not been submitted for the award of another degree or diploma at this or another institution. This declaration is subject to the candidate's review and signature before submission.

Place: Vellore

Date: _____

Signature of the candidate: _____

M Janani

25MDT0227

CERTIFICATE

This is to certify that the dissertation entitled "FireSense AI Fire and Smoke Detection Using YOLOv8", submitted by M Janani, registration number 25MDT0227, towards the degree of Master of Science in Data Science at VIT, presents the project work described in this document under the guidance of PRAVEEN T, Assistant Professor Grade 2, School of Advanced Sciences.

The certificate and examination details below are reserved for institutional review and approval. No signature, approval date or examination outcome is asserted in this unsigned copy.

Guide signature: _____

PRAVEEN T

Assistant Professor Grade 2

School of Advanced Sciences

Head of Department signature: _____

Internal examiner: _____

External examiner: _____

Date of viva voce examination: _____

ACKNOWLEDGEMENT

I thank my guide, PRAVEEN T, Assistant Professor Grade 2, School of Advanced Sciences, VIT, for the opportunity to present this project under his guidance.

I acknowledge Bo Peng and Tae-Kook Kim for releasing the Home Fire Dataset, and the developers of Ultralytics and the open-source libraries used in the implementation. Their published work and software made it possible to build and evaluate the detector with a traceable technical basis.

The project records both successful detections and the cases that required further work. Keeping those results visible has been an important part of preparing this dissertation.

M Janani

25MDT0227

ABSTRACT

FireSense AI is a local fire and smoke detection project built around YOLOv8. It provides dataset preparation, training, evaluation and a browser interface for camera frames, uploaded videos and still images. The project uses the publicly available Home Fire Dataset under CC BY-NC 4.0. An audit retained 6465 of its 6500 images, including 3873 training, 1297 validation and 1295 test images. The test split contains 1581 annotated objects across two classes.

A YOLOv8 nano baseline completed 20 training epochs, while a small-model baseline completed eight under the available compute and time budget. The nano checkpoint was refined for four additional epochs and then adapted for three epochs using 600 close-up and portrait views derived only from the training split. This adaptation addressed a supplied video containing large flames under strong red lighting. The video influenced candidate selection and is therefore treated as development evidence.

The selected model achieved 83.74% mAP@0.5 and 48.35% mAP@0.5:0.95 on the retained test split. At a minimum score of 0.50, precision was 93.04% and recall was 68.50%, with 1083 true positives, 81 false positives and 498 false negatives. Retained fire coverage improved from two to eight of eight fixed development frames. The interface displays raw model scores and supports annotated PNG and silent WebM downloads.

The completed project is a runnable academic prototype with saved checkpoints, curves, prediction examples and commands. Its missed-object rate, uncertain source-level split independence and limited negative-video testing prevent a claim of dependable unattended fire warning. The dissertation documents those limits and proposes source-independent video evaluation before wider use.

Keywords: fire detection, smoke detection, YOLOv8, object detection, local inference, domain adaptation.

TABLE OF CONTENTS

1 INTRODUCTION 1

1.1 Project context 1

1.2 Problem statement 1

1.3 Motivation and intended use 2

1.4 Related work and technical basis 2

1.5 Structure of the dissertation 2

2 PROJECT DESCRIPTION AND GOALS 4

2.1 Objective 4

2.2 Starting state and completed work 4

2.3 Functional requirements 5

2.4 Quality and acceptance criteria 5

2.5 Scope and deliverables 6

3 TECHNICAL SPECIFICATIONS 7

3.1 Compute and software environment 7

3.2 Public dataset and license 7

3.3 Data audit and preparation 8

3.4 Label representation and class balance 9

3.5 Repository and evidence organization 10

4 DESIGN APPROACH AND DETAILS 11

4.1 System architecture 11

4.2 YOLOv8 model design 12

4.3 Training and selection workflow 13

4.4 Baseline training configuration 14

4.5 Refinement and close-up adaptation 15

4.6 Browser inference and media output 16

4.7 Input boundaries and privacy 18

4.8 Evaluation method and metrics 18

5 SCHEDULE TASKS AND MILESTONES 19

5.1 Completed implementation sequence 19

5.2 Decisions made under resource limits 19

5.3 Completion checks 20

6 PROJECT OUTPUTS 21

6.1 Model and experiment artifacts 21

6.2 User interface 21

6.3 Annotated media downloads 22

6.4 Verification evidence 24

7 RESULTS AND DISCUSSION 25

7.1 Current held-out test results 25

7.2 Precision recall curve 26

7.3 Confusion matrix 27

7.4 Confidence threshold analysis 28

7.5 Model comparison 29

7.6 Training and validation observations 30

7.7 Correct detections and failure examples 31

7.8 Development video investigation 33

7.9 What the results mean for the project 34

8 LIMITATIONS 35

8.1 Dataset and evaluation limits 35

8.2 Model comparison limits 35

8.3 Runtime and usability limits 36

8.4 Safety and deployment limits 36

9 CONCLUSION 37

9.1 Completed outcome 37

9.2 Recommended next work 37

10 REFERENCES 38

APPENDIX A RUN INSTRUCTIONS 39

A.1 Environment and application 39

A.2 Preparation training and inference 39

A.3 Evaluation and checks 40

APPENDIX B EVIDENCE AND PROVENANCE 41

B.1 Dataset and training records 41

B.2 Metrics and model identity 41

B.3 Development and interface records.....42

B.4 Document reproduction 42

APPENDIX C HELD OUT TEST PREDICTION CASES.....44

LIST OF FIGURES

Figure 1 Usable image counts after the dataset audit. The retained source split contains 3873 training images, 1297 validation images and 1295 test images. Source evidence data_audit.json. 8
Figure 2 Fire and smoke annotation counts in the usable source splits. These are object counts rather than the number of images containing each class. 9
Figure 3 System architecture of the delivered local application. Model development produces a selected checkpoint and profile; browser requests use that profile through the local inference API. 11
Figure 4 Conceptual YOLOv8 detection path for the two-class task. The diagram describes the standard model components used here and is not a claim that a new detector architecture was invented. 12
Figure 5 Training and evaluation workflow. The development video influences candidate selection but supplies no training frames. The held-out test split supplies the reported benchmark metrics. 13
Figure 6 Training losses across the original nano run's 20 completed epochs. Loss terms have different meanings and should not be compared as a common physical quantity. Source runs/train/yolov8n/results.csv. 15
Figure 7 Validation AP trends during the original nano run. These are validation measurements during training, not held-out test scores. 15
Figure 8 Runtime flow for camera, video and image sources. Inference samples decoded frames; the interface does not claim to analyse every source frame. Recording finalizes on stop or video end. 17
Figure 9 Current local dashboard with source controls, detection preview and informative status. This is an actual browser capture from the completed interface, cropped for readability. 22
Figure 10 Downloaded annotated PNG from the interface. The exported file contains the detection overlay rather than only the original input image. 23
Figure 11 Decoded frame from the processed portrait-video download. The visible fire box and raw score confirm that the annotation is present in the exported media. 24

Figure 12 Native precision-recall curves for the selected model on the test split. Fire AP at IoU 0.50 is 0.833 and smoke AP is 0.842, giving mean AP of about 0.837. Source saved domain test run. 26
Figure 13 Native Ultralytics confusion matrix for the current test run. Its effective score threshold is 0.25 and its box-matching IoU is 0.45. These defaults differ from the fixed-threshold tables, which use score 0.50 and match IoU 0.50. 27
Figure 14 Precision and recall at four measured score filters on the same test set. These are discrete operating points, not a confidence-calibration curve. 28
Figure 15 Model comparison on the retained test split at 416 pixels. Training budgets differ. The small model stopped after eight epochs, so this figure does not establish that nano would win under equal training conditions. 29
Figure 16 Three successful current-model test examples at score 0.50. Left panels show dataset annotations; right panels show model predictions. Green denotes fire and blue denotes smoke. Examples are spaced through the list of images with no counted object errors. 31
Figure 17 Current-model test images with the highest combined false-positive and false-negative object counts at score 0.50 and matching IoU 0.50. Ground truth is shown alongside predictions so missing and extra boxes can be inspected. 32
Figure 18 Same development frame at 9.6 seconds before and after adaptation, using a 0.50 score filter. The previous refined model has no retained box here; the current model retains a fire box. Both panels use the identical source image. 33
Figure 19 Test case 01, test_3.jpg. Ground truth and current model predictions at score 0.50. Counts use class-aware matching at IoU 0.50. 45
Figure 20 Test case 02, test_95.jpg. Ground truth and current model predictions at score 0.50. Counts use class-aware matching at IoU 0.50. 46
Figure 21 Test case 03, test_191.jpg. Ground truth and current model predictions at score 0.50. Counts use class-aware matching at IoU 0.50. 47
Figure 22 Test case 04, test_295.jpg. Ground truth and current model predictions at score 0.50. Counts use class-aware matching at IoU 0.50. 48
Figure 23 Test case 05, test_381.jpg. Ground truth and current model predictions at score 0.50. Counts use class-aware matching at IoU 0.50. 49
Figure 24 Test case 06, test_481.jpg. Ground truth and current model predictions at score 0.50. Counts use class-aware matching at IoU 0.50. 50

Figure 25 Test case 07, test_572.jpg. Ground truth and current model predictions at score 0.50. Counts use class-aware matching at IoU 0.50. 51
Figure 26 Test case 08, test_656.jpg. Ground truth and current model predictions at score 0.50. Counts use class-aware matching at IoU 0.50. 52
Figure 27 Test case 09, test_738.jpg. Ground truth and current model predictions at score 0.50. Counts use class-aware matching at IoU 0.50. 53
Figure 28 Test case 10, test_835.jpg. Ground truth and current model predictions at score 0.50. Counts use class-aware matching at IoU 0.50. 54
Figure 29 Test case 11, test_923.jpg. Ground truth and current model predictions at score 0.50. Counts use class-aware matching at IoU 0.50. 55
Figure 30 Test case 12, test_1019.jpg. Ground truth and current model predictions at score 0.50. Counts use class-aware matching at IoU 0.50. 56
Figure 31 Test case 13, test_1108.jpg. Ground truth and current model predictions at score 0.50. Counts use class-aware matching at IoU 0.50. 57
Figure 32 Test case 14, test_1209.jpg. Ground truth and current model predictions at score 0.50. Counts use class-aware matching at IoU 0.50. 58
Figure 33 Test case 15, test_1300.jpg. Ground truth and current model predictions at score 0.50. Counts use class-aware matching at IoU 0.50. 59
Figure 34 Test case 16, test_288.jpg. Ground truth and current model predictions at score 0.50. Counts use class-aware matching at IoU 0.50. 60
Figure 35 Test case 17, test_255.jpg. Ground truth and current model predictions at score 0.50. Counts use class-aware matching at IoU 0.50. 61
Figure 36 Test case 18, test_908.jpg. Ground truth and current model predictions at score 0.50. Counts use class-aware matching at IoU 0.50. 62
Figure 37 Test case 19, test_15.jpg. Ground truth and current model predictions at score 0.50. Counts use class-aware matching at IoU 0.50. 63
Figure 38 Test case 20, test_126.jpg. Ground truth and current model predictions at score 0.50. Counts use class-aware matching at IoU 0.50. 64
Figure 39 Test case 21, test_221.jpg. Ground truth and current model predictions at score 0.50. Counts use class-aware matching at IoU 0.50. 65
Figure 40 Test case 22, test_327.jpg. Ground truth and current model predictions at score 0.50. Counts use class-aware matching at IoU 0.50. 66

Figure 41 Test case 23, test_438.jpg. Ground truth and current model predictions at score 0.50. Counts use class-aware matching at IoU 0.50. 67
Figure 42 Test case 24, test_544.jpg. Ground truth and current model predictions at score 0.50. Counts use class-aware matching at IoU 0.50. 68
Figure 43 Test case 25, test_687.jpg. Ground truth and current model predictions at score 0.50. Counts use class-aware matching at IoU 0.50. 69
Figure 44 Test case 26, test_794.jpg. Ground truth and current model predictions at score 0.50. Counts use class-aware matching at IoU 0.50. 70
Figure 45 Test case 27, test_940.jpg. Ground truth and current model predictions at score 0.50. Counts use class-aware matching at IoU 0.50. 71
Figure 46 Test case 28, test_1082.jpg. Ground truth and current model predictions at score 0.50. Counts use class-aware matching at IoU 0.50. 72
Figure 47 Test case 29, test_1175.jpg. Ground truth and current model predictions at score 0.50. Counts use class-aware matching at IoU 0.50. 73
Figure 48 Test case 30, test_1290.jpg. Ground truth and current model predictions at score 0.50. Counts use class-aware matching at IoU 0.50. 74

LIST OF TABLES

Table 1 Analysis of the starting state and completed work 4
Table 2 Recorded software and compute environment 7
Table 3 Dataset split and annotation inventory 8
Table 4 Repository responsibilities 10
Table 5 Original model training configuration 14
Table 6 Domain adaptation training configuration 16
Table 7 Completed implementation stages 19
Table 8 Model and deployment artifacts 21
Table 9 Current held-out test results 25
Table 10 Measured confidence operating points 28
Table 11 Model comparison with completed training budgets 29
Table 12 Adaptation epoch log 30
Table 13 Development-video observations 34
Table 14 Dataset and training evidence 41
Table 15 Metrics and checkpoint evidence 41
Table 16 Development and interface evidence 42

LIST OF ABBREVIATIONS

AI Artificial intelligence

AP Average precision

API Application programming interface

CNN Convolutional neural network

COCO Common Objects in Context

CUDA Compute Unified Device Architecture

DFL Distribution focal loss

FP FN TP False positive, false negative and true positive

FP32 32-bit floating point

GPU Graphics processing unit

HSV Hue saturation and value

HTTP Hypertext Transfer Protocol

IoU Intersection over union

mAP Mean average precision

NMS Non-maximum suppression

PNG Portable Network Graphics

RGB Red green and blue

SPPF Spatial pyramid pooling fast

WCAG Web Content Accessibility Guidelines

YOLO You Only Look Once

1 INTRODUCTION

1.1 Project context

FireSense AI is a local computer vision application that identifies visible fire and smoke in images, uploaded videos and camera frames. It combines a trained YOLOv8 detector with a browser interface, saved evaluation results and downloadable annotated media. The completed system uses a domain-adapted YOLOv8 nano model. On the retained test split, it achieved 83.74% mAP@0.5 and 48.35% mAP@0.5:0.95. At a detection score threshold of 0.50, precision was 93.04% and recall was 68.50%.

These results support a practical demonstration of visual detection, with clear limits. A model may overlook a flame, draw an inaccurate boundary, or mistake a visually similar region for fire. Smoke is especially difficult when its boundary blends into the background. The application therefore presents model output for review. It has not been validated as a certified fire alarm or as a replacement for installed safety equipment.

The project began with a research proposal rather than an evaluated implementation. The proposal described fire detection and possible response functions. The work documented here turns the detection component into a runnable pipeline, measures its performance, and records the changes needed when a close-up video exposed a weakness. The final interface supports camera, video and image input. Physical suppression, email alerts and virtual sprinkler simulation are outside the delivered application.

1.2 Problem statement

A detector that performs reasonably on a benchmark can still fail on a new video. Camera viewpoint, lighting, object size and the surrounding scene all affect the features seen by the model. In this project, a portrait video containing large flames under strong red illumination produced few retained detections with the earlier checkpoint. Raising a displayed confidence value would not have corrected that failure. The model needed better training coverage of the observed conditions, followed by checks that ordinary validation performance had not deteriorated.

The problem addressed by this dissertation is to build a reproducible fire and smoke detector that can be used through a local browser, while keeping its measured capabilities and limitations visible. This requires more than a successful training command. Dataset

provenance, label checks, model selection, held-out evaluation and the behaviour of the media interface all affect whether the output can be interpreted correctly.

1.3 Motivation and intended use

Visual detection can help a person review a scene by marking likely fire and smoke regions. Bounding boxes make the output easier to inspect than a single image-level warning. A small detector is also suitable for the available laptop GPU and for repeated local requests from a browser. The intended setting is an academic prototype for observation and experimentation, including analysis of uploaded media and a local camera feed.

The interface keeps the source visible, overlays the retained predictions and allows the user to save an annotated frame or processed clip. This makes errors easier to notice and discuss. It also provides evidence that can be included in a report without relying on a screenshot of a console. The exported media remains an illustration of model behaviour; it does not add ground-truth labels or certify the scene as safe.

1.4 Related work and technical basis

The original YOLO work describes object detection as a single network prediction problem, with bounding boxes and class information produced from an image [1]. That approach provides the background for choosing a detector that can serve repeated requests. This project uses the Ultralytics YOLOv8 implementation, whose detection models are available in several sizes [2]. The nano and small variants provide a useful comparison under the available compute budget.

The Home Fire Dataset was released by Bo Peng and Tae-Kook Kim for home fire research [3]. Its associated YOLO-HF paper provides the dataset context [4]. This project trains the standard YOLOv8 implementation on that dataset; it does not claim to reproduce the authors' YOLO-HF architecture or their published experimental results. COCO-pretrained weights supply the initial parameters [5]. The fire and smoke labels and every metric in this dissertation come from the local project experiments.

1.5 Structure of the dissertation

The following chapters define the project goals, describe the software and data, and explain the design. The implementation stages are presented as completed tasks because the available evidence does not establish a semester-long calendar. The results chapter separates benchmark

detection metrics from the development-video investigation. The final chapters discuss limits, recommendations and the files needed to reproduce the work.

2 PROJECT DESCRIPTION AND GOALS

2.1 Objective

The objective is to detect and localize two visible classes, fire and smoke, using a trained YOLOv8 model and to make that model accessible through a local application. The full workflow covers public dataset preparation, training, comparison, evaluation, inference and export. A reader should be able to identify the active checkpoint, repeat a documented command and trace a reported value to a saved result file.

The project does not assume that a high confidence score guarantees a correct detection. Confidence is used to filter model predictions. Correctness is assessed by comparing predicted classes and boxes with annotations at a stated intersection-over-union threshold. This distinction is necessary because the request for scores above 90% initially obscured a recall problem on the supplied video.

2.2 Starting state and completed work

The starting artifact was a research proposal. There was no measured training run or completed detection interface to substantiate its intended behaviour. The existing project now contains a data preparation pipeline, separate training and evaluation commands, saved checkpoints, experiment evidence, a local inference server and a browser application. Subsequent changes addressed incorrect detections on the supplied development video and added processed-media downloads.

Table 1 Analysis of the starting state and completed work

Area	Starting state	Completed implementation
Data	Public dataset not yet prepared	Source record, label audit, duplicate checks and split manifest
Model	YOLOv8 proposed	Nano and small baselines, nano refinement and domain adaptation
Evaluation	No measured baseline	Test AP, fixed-threshold counts, curves and error examples

Area	Starting state	Completed implementation
Interface	No verified end-to-end flow	Local camera, video and image analysis with visible boxes
Output	No processed-media export	Annotated PNG and silent WebM downloads
Response	Wider proposal ambitions	Detection only; response hardware and simulator excluded

2.3 Functional requirements

The application must accept a still image, an uploaded video or an authorized browser camera stream. It must display the source and the retained detections using the same image coordinate system. A user must be able to change the minimum score, stop an ongoing source and save supported output. Training, inference and evaluation must remain available independently of the interface so that model work can be repeated from a terminal.

The model pipeline must preserve the meaning of the two dataset classes. It must reject malformed labels and prevent validation or test images from being added to the training augmentation pool. Evaluation must state both the score filter and the box-matching rule. The active model profile must identify the selected weights and settings rather than silently choosing whichever checkpoint was created most recently.

2.4 Quality and acceptance criteria

The domain-adaptation acceptance contract was recorded before the final candidate evaluation. It required validation mAP@0.5:0.95 of at least 0.55, precision of at least 0.90 at the 0.50 score threshold, and retained fire boxes that visually aligned with the flames in all eight fixed development frames. The contract permitted a disclosed validation localization reduction of about two percentage points if needed for close-up recovery. The selected model met the contract without that reduction.

These criteria concern the available validation set and a known development case. They do not establish a general field performance target. The final test split supplies a separate benchmark description, while browser checks establish that input, requests, overlays and downloads work

together. No minimum certified alarm recall or maximum permitted safety response delay has been demonstrated.

2.5 Scope and deliverables

The delivered files include the prepared dataset configuration, model checkpoints, training logs, machine-readable metrics, prediction examples, the local dashboard and this dissertation. The original and refined weights are retained for comparison and rollback. The public dataset license limits use to the terms of CC BY-NC 4.0; the presence of a working application does not remove those obligations [3, 6].

The source video supplied for diagnosis was not used as training data. It was inspected and used during candidate selection, so it is described as development data throughout this document. This is a deliberate boundary in the evaluation claim. A future test of generalization needs additional videos that have not influenced model development.

3 TECHNICAL SPECIFICATIONS

3.1 Compute and software environment

Training used the available NVIDIA GTX 1650 GPU with 4 GB of memory on Windows 11. The nano model was chosen as the main candidate because it fit the available memory and allowed more training and local inference work within the project budget. Full precision was used. The environment uses pinned versions because newer combinations of PyTorch and numerical libraries encountered Windows application-control or binary loading problems during setup.

Table 2 Recorded software and compute environment

Component	Recorded configuration
Operating system	Windows 11
Python	3.12.14 in the project virtual environment
GPU	NVIDIA GTX 1650 with 4 GB memory
PyTorch	2.5.1 with CUDA 12.1 runtime
Ultralytics	8.3.221
NumPy and OpenCV	1.26.4 and 4.10.0
Inference image size	416 pixels with model preprocessing
Application	Python local HTTP server and HTML CSS JavaScript

The requirements file records the compatible dependencies. A CPU can be selected for the dashboard, but no equivalent CPU responsiveness claim is made here. The measured batch evaluation timing is specific to the recorded hardware and run. It includes neither browser decoding nor transfer and rendering time, so it should not be converted into an end-to-end live frame-rate promise.

3.2 Public dataset and license

The Home Fire Dataset version 1.0.0 contains 6,500 published images in YOLO format, with fire as class 0 and smoke as class 1. The source provides 3,900 training, 1,300 validation and 1,300 test images [3]. The project retained that split structure rather than creating a new random split. After auditing, 6,465 images remained usable.

The source repository identifies the license as Creative Commons Attribution-NonCommercial 4.0. The source, authors, dataset version and license must accompany reuse. The dataset should not be assumed suitable for unrestricted commercial deployment. Any future product release must review the dataset and software license obligations together.

Table 3 Dataset split and annotation inventory

Split	Published images	Usable images	Fire boxes	Smoke boxes	Background images
Training	3900	3873	2951	1834	87
Validation	1300	1297	960	617	33
Test	1300	1295	892	689	15
Total	6500	6465	4803	3140	135

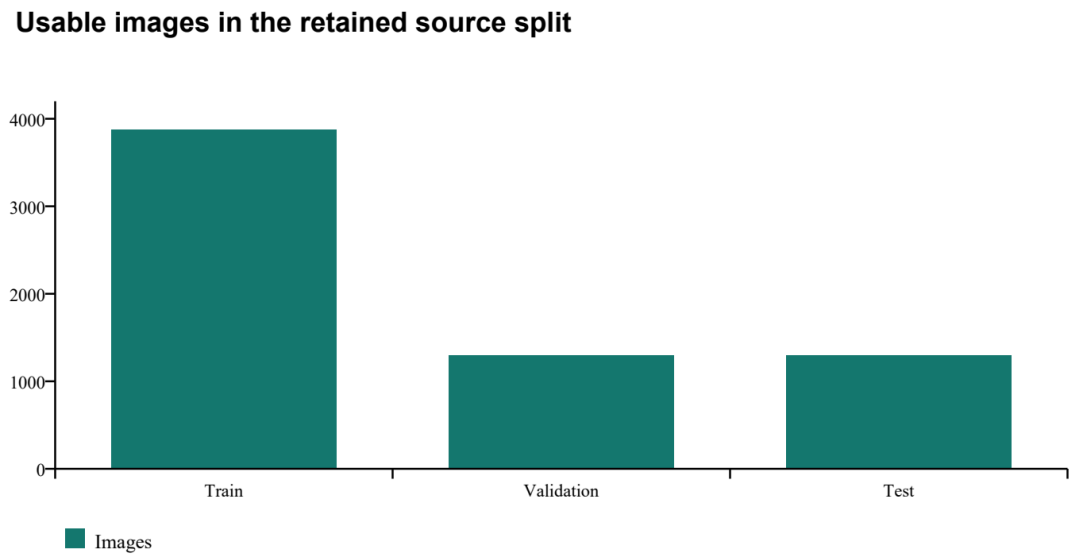


Figure 1 Usable image counts after the dataset audit. The retained source split contains 3873 training images, 1297 validation images and 1295 test images. Source evidence data_audit.json.

3.3 Data audit and preparation

The preparation code validates the class identifier and normalized box coordinates before using a label. Coordinates must be finite, dimensions must be positive, and boxes must lie within the image. Images are decoded into RGB and resized to a maximum side of 640 pixels while their aspect ratio is preserved. Prepared JPEG images use quality 95. The labels remain normalized to the prepared image dimensions.

Thirty-three images were excluded because EXIF orientation would have required explicit conversion of their annotations. Two exact duplicates were removed using a hash of decoded RGB pixels. Both duplicates occurred within the test split and had slightly different annotation boxes. The audit therefore avoids counting identical pixels twice, but it does not establish that all remaining scenes are independent.

The duplicate procedure checks splits in test, validation and training priority order. It protects the retained test copy when an exact match is encountered elsewhere. No source-video identifiers were available to prove that visually related frames never cross splits. The test results must be read with that uncertainty in mind. Near duplicates and shared scene content remain possible even after exact pixel hashing.

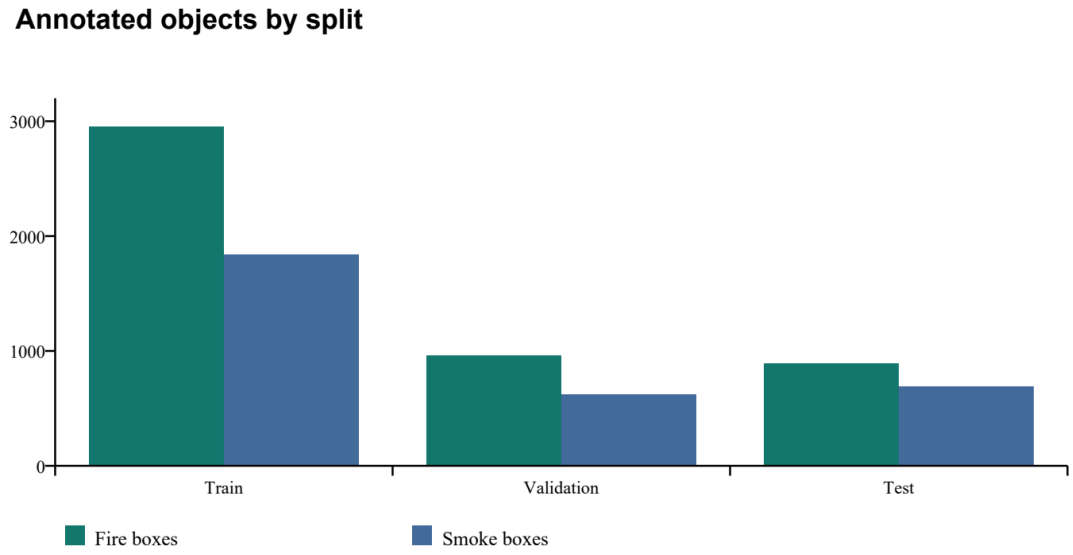


Figure 2 Fire and smoke annotation counts in the usable source splits. These are object counts rather than the number of images containing each class.

3.4 Label representation and class balance

Each YOLO label row stores class, horizontal centre, vertical centre, width and height. The last four values are normalized by image dimensions. Evaluation converts them to pixel corners and compares them with predicted boxes. This explicit conversion is also used when checking predictions and rendering ground truth, which reduces the chance that a visually plausible overlay conceals a coordinate mismatch.

The training split has more fire boxes than smoke boxes and relatively few background-only images. The test split contains just 15 background images. That limits what can be inferred about false alarms in ordinary non-fire scenes. Precision measured on this dataset is useful for

comparison within the project, but a long, mostly negative camera recording could expose errors that are poorly represented here.

3.5 Repository and evidence organization

Table 4 Repository responsibilities

Location	Purpose
pipeline.py	Dataset preparation, baseline training and command-line prediction
evaluate.py	Baseline evaluation and class-aware box matching
scripts/optimize_detector.py	Current checkpoint validation and test metrics at selected image sizes
scripts/adapt_detector.py	Training-only close-up augmentation and candidate training
dashboard.py	Local inference service and application asset delivery
web/	Browser source controls, preview, recording and download logic
models/ and runs/	Checkpoints, arguments, epoch logs and native plots
evidence/	Data audits, selection contracts, metrics and browser checks
reports/	Experimental reports and dissertation source

The evidence directory is part of the result, not an optional supplement. It records the selected model hash, metric denominators, augmentation sources and browser observations. Some large data and training files are ignored by version control but remain in the local project. Reproducing the project on a different machine requires those files or a fresh dataset download and training run.

4 DESIGN APPROACH AND DETAILS

4.1 System architecture

The system separates offline model development from interactive inference. The offline pipeline prepares data, trains checkpoints and writes evaluation evidence. The local service loads the active deployment profile and accepts a validated image request. The browser decodes the selected source, captures a suitably sized frame and draws the returned boxes over its preview.

This separation keeps the training environment independent of the interface. It also makes the saved weights and selection profile explicit. A change to the browser layout does not retrain the model, and a new checkpoint can be evaluated without opening the interface. The architecture diagram shows the paths used by the completed system; no cloud detection service is involved.

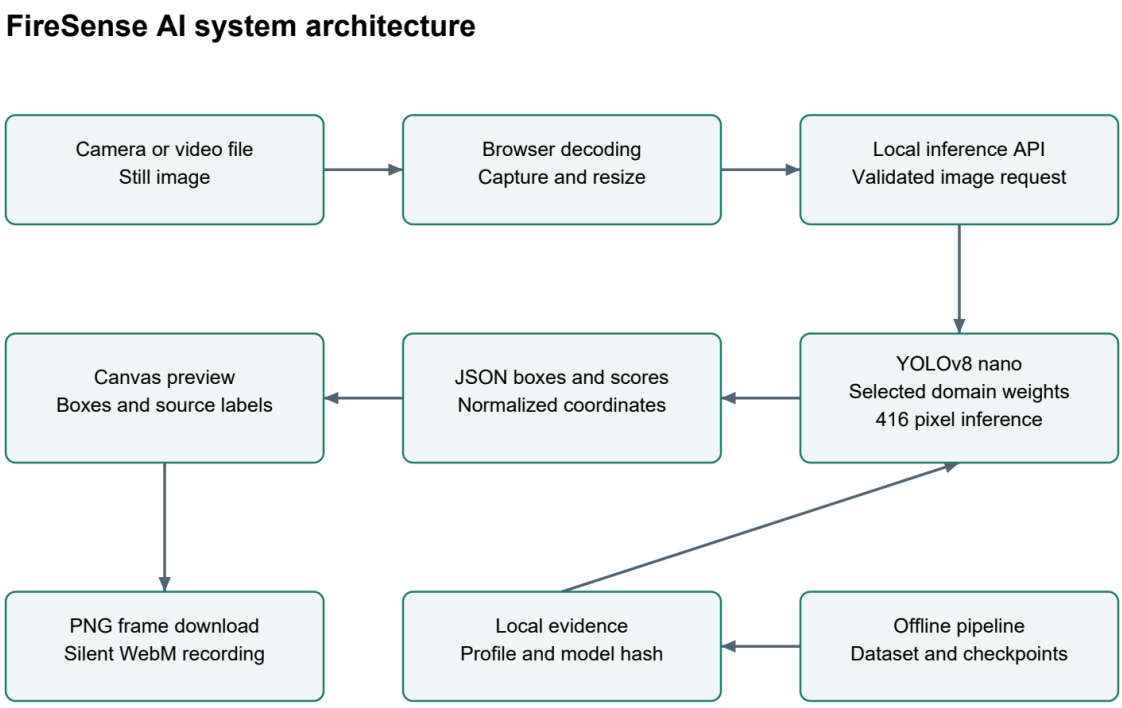


Figure 3 System architecture of the delivered local application. Model development produces a selected checkpoint and profile; browser requests use that profile through the local inference API.

4.2 YOLOv8 model design

YOLOv8 uses convolutional feature extraction, C2f blocks, spatial pyramid pooling and a detection head operating at multiple feature scales [2]. The neck combines information from different resolutions. The project configures the detection head for two classes. The network predicts candidate boxes and scores, after which postprocessing filters scores and applies non-maximum suppression.

The model input size is 416 pixels. Model preprocessing handles resizing and padding before inference, while the browser may first reduce the source frame for transfer. Returned coordinates are normalized to the submitted image and are mapped to the displayed preview. This mapping is necessary for portrait videos and other sources whose dimensions differ from the model input.

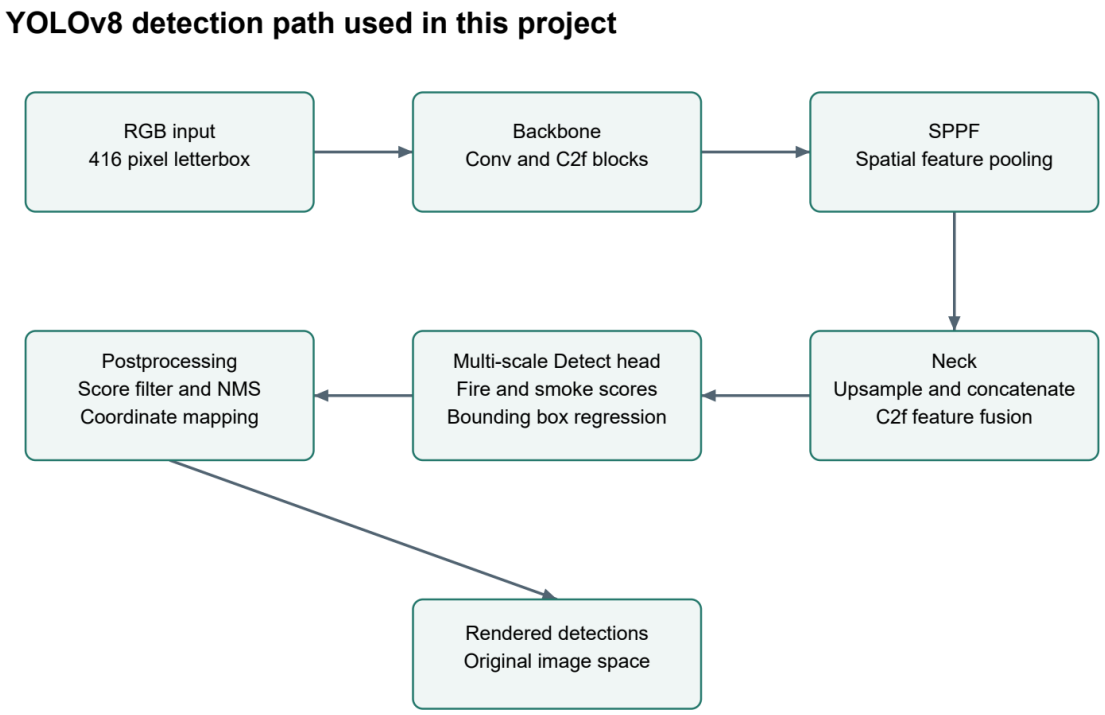


Figure 4 Conceptual YOLOv8 detection path for the two-class task. The diagram describes the standard model components used here and is not a claim that a new detector architecture was invented.

The nano checkpoint contains approximately 3.01 million parameters; the small checkpoint contains approximately 11.13 million. Their saved baseline weight files occupy about 6.20 MB and 22.49 MB respectively. Parameter count and file size describe resource demand, while

detection quality must be measured separately. In this project the models received unequal training budgets, which limits the architectural comparison.

4.3 Training and selection workflow

The original training split is the only source for parameter fitting. Validation supports checkpoint and configuration choices. The test split is evaluated to report performance after a candidate is chosen. The original nano and small runs establish a baseline, followed by a lower-learning-rate nano refinement. A later adaptation stage adds close-up and portrait variations derived from training images to address the development-video failure.

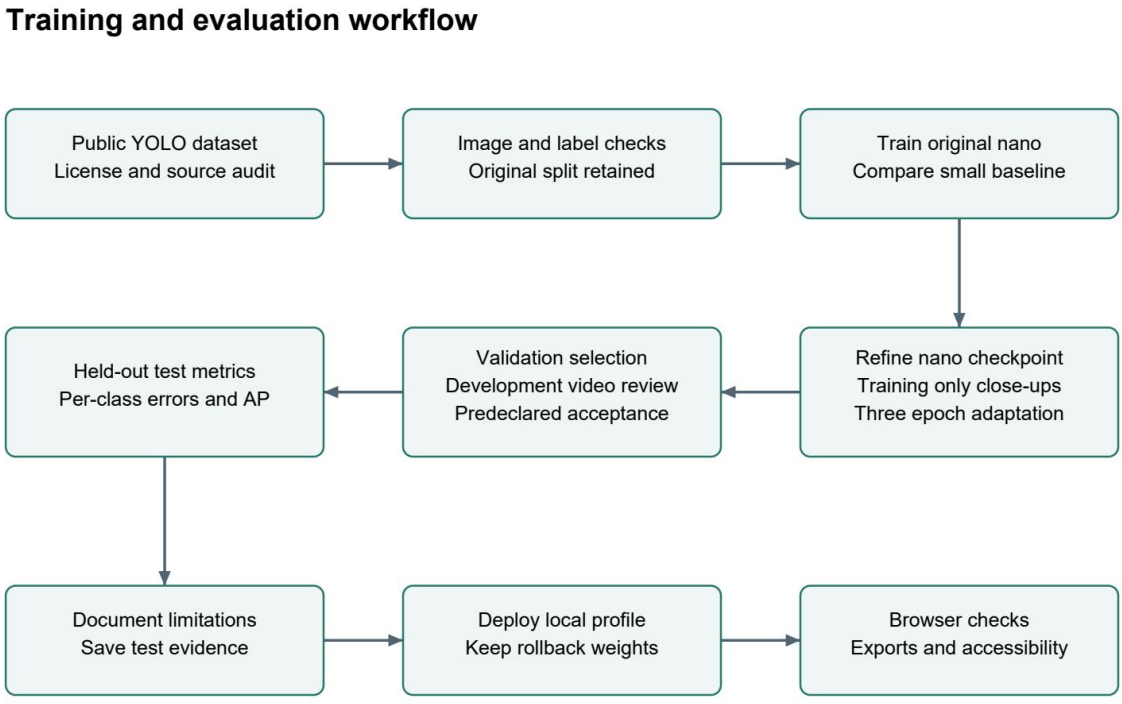


Figure 5 Training and evaluation workflow. The development video influences candidate selection but supplies no training frames. The held-out test split supplies the reported benchmark metrics.

Validation and test images are not used to generate the 600 additional training views. The augmentation manifest records the source image and crop for every derived view. The final training list contains 4,473 entries: 3,873 original images and 600 derived views. It should not be described as 4,473 independently collected images.

4.4 Baseline training configuration

Both baseline models start from COCO-pretrained YOLOv8 weights. Training uses AdamW, an initial learning rate of 0.001, weight decay of 0.0005 and a fixed seed of 42. The recorded nano run completed 20 epochs. The small run completed eight of its planned 20 epochs before it was stopped to meet the request for faster completion. The best saved checkpoint from each completed run is used for the baseline comparison.

Table 5 Original model training configuration

Setting	Original nano and small runs
Image size and batch	416 pixels and batch 4
Completed epochs	Nano 20 and small 8
Optimizer	AdamW with lr0 0.001 and lrf 0.01
Regularization	Weight decay 0.0005 and warmup 2 epochs
Precision and workers	FP32 with workers 0
Geometry	Scale 0.5, translate 0.1 and horizontal flip 0.5
Colour	HSV gains h 0.015, s 0.7 and v 0.4
Mixed images	Mosaic 1.0, disabled for the final 3 epochs; mixup 0
Reproducibility	Seed 42, deterministic setting and patience 8

The epoch CSV files retain training losses and validation metrics. The arguments saved with each run remain the authoritative configuration. A fixed seed improves repeatability, but numerical results can still vary with GPU kernels, package versions or a changed dataset preparation path. The saved checkpoint and its hash identify the actual model used for the reported inference.

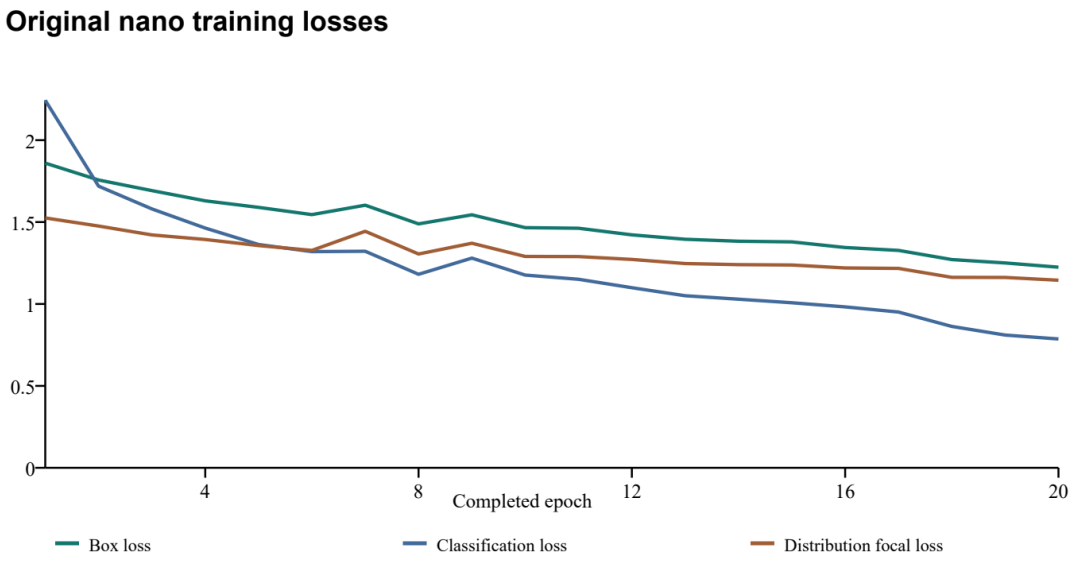


Figure 6 Training losses across the original nano run's 20 completed epochs. Loss terms have different meanings and should not be compared as a common physical quantity. Source runs/train/yolov8n/results.csv.

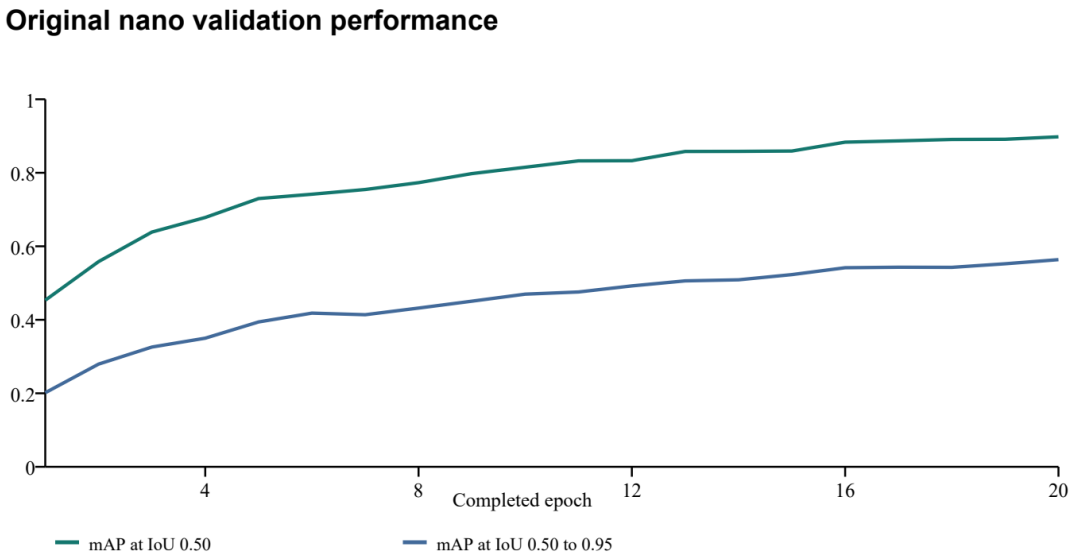


Figure 7 Validation AP trends during the original nano run. These are validation measurements during training, not held-out test scores.

4.5 Refinement and close-up adaptation

The first nano refinement trained for four additional epochs from the baseline checkpoint. It used a learning rate of 0.00015, batch 16, disabled mosaic and mixup, and froze the first ten backbone layers. Geometry and colour changes were reduced. This stage improved the measured test localization AP compared with the original nano model, but it still failed to retain fire boxes in most fixed frames of the supplied portrait video.

The adaptation stage starts from the refined weights and trains for three more epochs. It freezes parameters in the first four layers and uses AdamW with an initial learning rate of 0.0002. Each stage starts a new optimizer schedule. The notation 20+4+3 therefore describes sequential training stages, not a single uninterrupted 27-epoch run.

Table 6 Domain adaptation training configuration

Setting	Domain adaptation
Training entries	3873 original images plus 600 training-only views
Image size, batch and epochs	416 pixels, batch 16 and 3 completed epochs
Optimizer	AdamW, lr0 0.0002, lrf 0.1 and weight decay 0.0005
Warmup and stopping	1 warmup epoch and patience 3
Geometry	Scale 0.15, translate 0.05 and horizontal flip 0.5
Colour	HSV gains h 0.005, s 0.2 and v 0.2
Mixed images	Mosaic 0 and mixup 0
Other settings	FP32, seed 42 and deterministic setting

The additional views crop around the largest annotated fire box with 10% to 30% context. Intersecting labels are clipped and remapped; boxes retaining less than 15% of their original visible area are discarded. The crop is resized to 360 by 640 pixels as a portrait augmentation. Half the views receive randomized channel gains that reduce blue and green relative to red. These transformations target the observed appearance without adding frames from the development video.

An initial batch-eight adaptation attempt was stopped before any epoch completed. It did not supply the selected checkpoint. The completed batch-16 run took about 481 seconds for three epochs according to its epoch log. That elapsed value describes this training stage only; it excludes data preparation, earlier training and evaluation.

4.6 Browser inference and media output

The browser uses decoded video-frame callbacks where supported and permits one inference request at a time. Rendering runs independently, so the preview is not intentionally stalled

while waiting for a model response. Analysis frames are resized within a 640 by 480 envelope while preserving aspect ratio. The preview can use a larger 1280 by 720 envelope. Different envelopes do not mean that inference runs at the original video resolution.

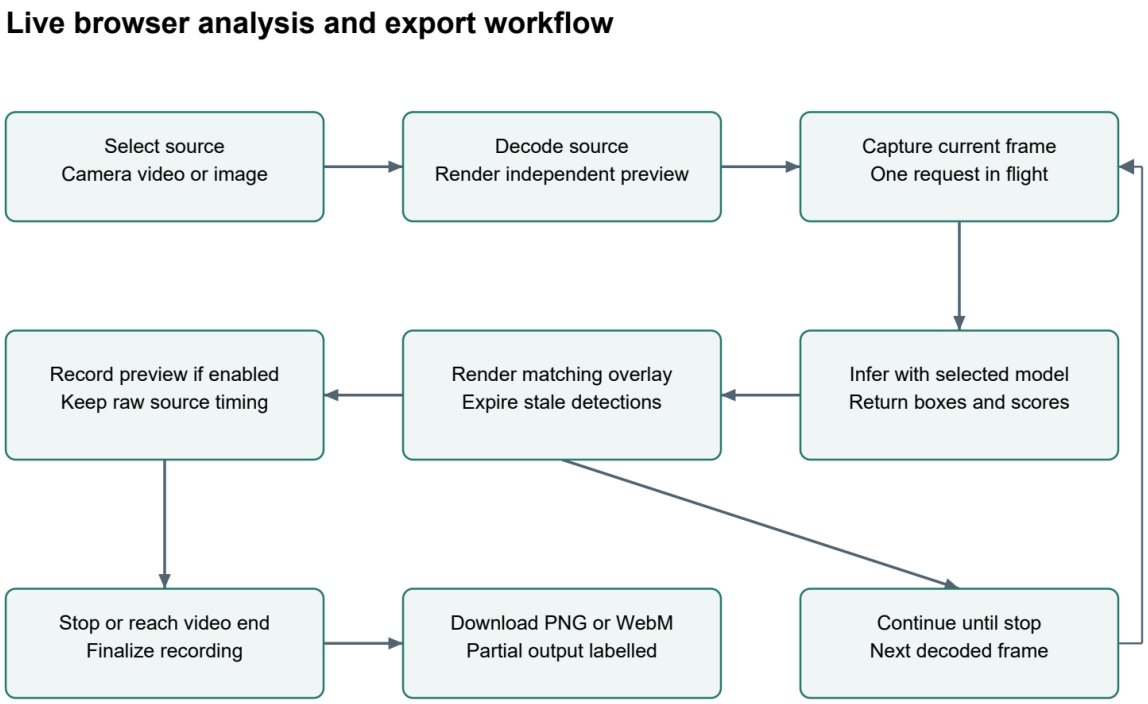


Figure 8 Runtime flow for camera, video and image sources. Inference samples decoded frames; the interface does not claim to analyse every source frame. Recording finalizes on stop or video end.

Predictions older than 750 milliseconds are hidden to reduce stale overlays. The default score threshold is 0.50. The interface also allows a stricter threshold, accompanied by a warning about missed objects. Displayed scores are the model's values; they are not increased to satisfy a desired percentage.

A still frame is exported as PNG with its current overlay. Video output is a silent WebM recording of the annotated preview, requested at 15 frames per second. Uploaded videos are recorded by default; camera recording requires the user's choice. The browser finalizes a recording at the end or on stop, labels partial recordings, and retains the previous completed clip until a new one replaces it. Memory and duration limits protect the local session from unbounded recording.

4.7 Input boundaries and privacy

The service binds to the local address 127.0.0.1. It checks host and origin, accepts bounded image requests and validates the decoded image before inference. The request limit is 3 MB with an 8-megapixel decoded-frame limit. A transfer time bound and an inference lock help keep malformed or overlapping requests from disrupting ordinary use. Static assets are served from an allowlist with a content security policy.

Camera frames and uploaded media are processed locally in this design. The application does not require an external inference account. Downloads are initiated by the user. This local arrangement reduces external data transfer, but it does not make a shared computer or a browser permission automatically private. A user should still stop the camera session and handle downloaded media according to the setting in which it was recorded.

4.8 Evaluation method and metrics

Intersection over union is the area shared by a predicted box and an annotated box divided by their union. The fixed-threshold evaluation sorts predictions by confidence, matches each prediction to an unused annotation of the same class, and accepts a match at IoU of at least 0.50. A matched prediction is a true positive. An unmatched prediction is a false positive, and an unmatched annotation is a false negative.

Precision = TP / (TP + FP). Recall = TP / (TP + FN). Overall precision and recall in the fixed-threshold tables are micro-averaged by summing counts over the two classes. Average precision summarizes the precision-recall relationship over confidence values. mAP@0.5 averages class AP at IoU 0.50. mAP@0.5:0.95 averages over stricter matching thresholds from 0.50 to 0.95 [7]. A broad but poorly aligned box may contribute at 0.50 and fail at higher IoU values.

Native validation runs use a low initial score cutoff of 0.001 and non-maximum suppression IoU of 0.70 to produce AP curves. The project then computes operating-point counts at 0.25, 0.50, 0.75 and 0.90 from saved predictions. Non-maximum suppression IoU, annotation-matching IoU and confidence are separate settings. The native confusion matrix uses its own defaults, stated with that figure in Chapter 7.

5 SCHEDULE TASKS AND MILESTONES

5.1 Completed implementation sequence

The available files establish an order of implementation and saved outcomes. They do not establish an approved academic weekly schedule, so this chapter reports stages and evidence rather than assigning invented dates. Each stage has a concrete artifact that can be checked independently of this narrative.

Table 7 Completed implementation stages

Stage	Completed task	Evidence
1	Review proposal and establish detection scope	Starting proposal and final project reports
2	Prepare and audit public data	data_audit.json and manifest.json
3	Train original nano and small baselines	Training arguments, CSV logs and best weights
4	Evaluate baselines and choose nano	Validation recommendation and test_metrics.json
5	Refine nano at a lower learning rate	nano_refinement2 logs and refined metrics
6	Reproduce close-up video failure	Eight fixed frames and baseline_raw_frames.json
7	Adapt using training-only variations	Augmentation manifest and adaptation log
8	Select candidate and report held-out metrics	Selection contract and domain evaluation JSON
9	Complete interface and media downloads	Browser checks, downloaded PNG and WebM
10	Assemble dissertation and figures	This document and its reproducible source

5.2 Decisions made under resource limits

The main resource constraint was the available GPU memory and the request to complete the work promptly. A nano model at 416 pixels permitted repeatable training and testing. The small

baseline was stopped after eight epochs, which saved time but weakened the fairness of the model-size comparison. That decision remains visible in the results table instead of being hidden behind a claim that one architecture is universally better.

The close-up problem was approached through diagnosis before additional training. Larger inference sizes and several image preprocessing choices were tested but did not solve the supplied case. The final change was a short adaptation stage with training-derived crops and colour variation. The project retained previous checkpoints so the selected model could be compared or rolled back without recreating the earlier experiments.

5.3 Completion checks

Completion means that preparation, training, prediction and evaluation have documented entry points; the local interface accepts supported sources; and the reported results have saved evidence. It does not mean every possible fire scene has been tested. The final verification includes 13 Python unit tests, four JavaScript export lifecycle tests and browser checks for source handling, media output and accessibility.

The browser accessibility record covers seven interface states using automated WCAG A and AA checks, including WCAG 2.1 AA rules and best-practice checks, with no recorded violations in those scans. Keyboard interactions and several viewport widths were checked. Automated scans cannot establish complete usability for every assistive technology. A physical webcam and a formal screen-reader study remain outside the recorded verification.

6 PROJECT OUTPUTS

6.1 Model and experiment artifacts

The deployed model is models/yolov8n_domain.pt. Its active profile is evidence/optimization_selection.json, which records input size 416, default confidence 0.50 and the checkpoint SHA256. The original nano, original small and refined nano files remain in models/. This makes the recommendation reviewable and avoids overwriting the only copy of an earlier model.

Table 8 Model and deployment artifacts

Artifact	Meaning
yolov8n_best.pt	Original nano checkpoint from the 20-epoch run
yolov8s_best.pt	Small checkpoint from eight completed epochs
yolov8n_refined.pt	Nano after four additional refinement epochs
yolov8n_domain.pt	Selected nano after three adaptation epochs
optimization_selection.json	Active model identity and default inference settings
runs/optimization/test_yolov8n_domain_416/	Native test curves, confusion matrix and predictions

The selected checkpoint hash is 4d9b2ce49bcf61694427e1e756b2e66ed5005d6552cb999950de2ceccdf64b41. A matching hash identifies the file that produced the reported deployment behaviour. It does not verify the correctness of the model by itself. The accompanying metrics and prediction examples provide that experimental context.

6.2 User interface

The final page uses a white theme with a source panel, readable settings, a large preview and separate download controls. The user can select a camera, upload a video or analyse an image. Source and model status remain visible so that a paused video, an unavailable camera or an active request is less likely to be mistaken for a detection outcome.

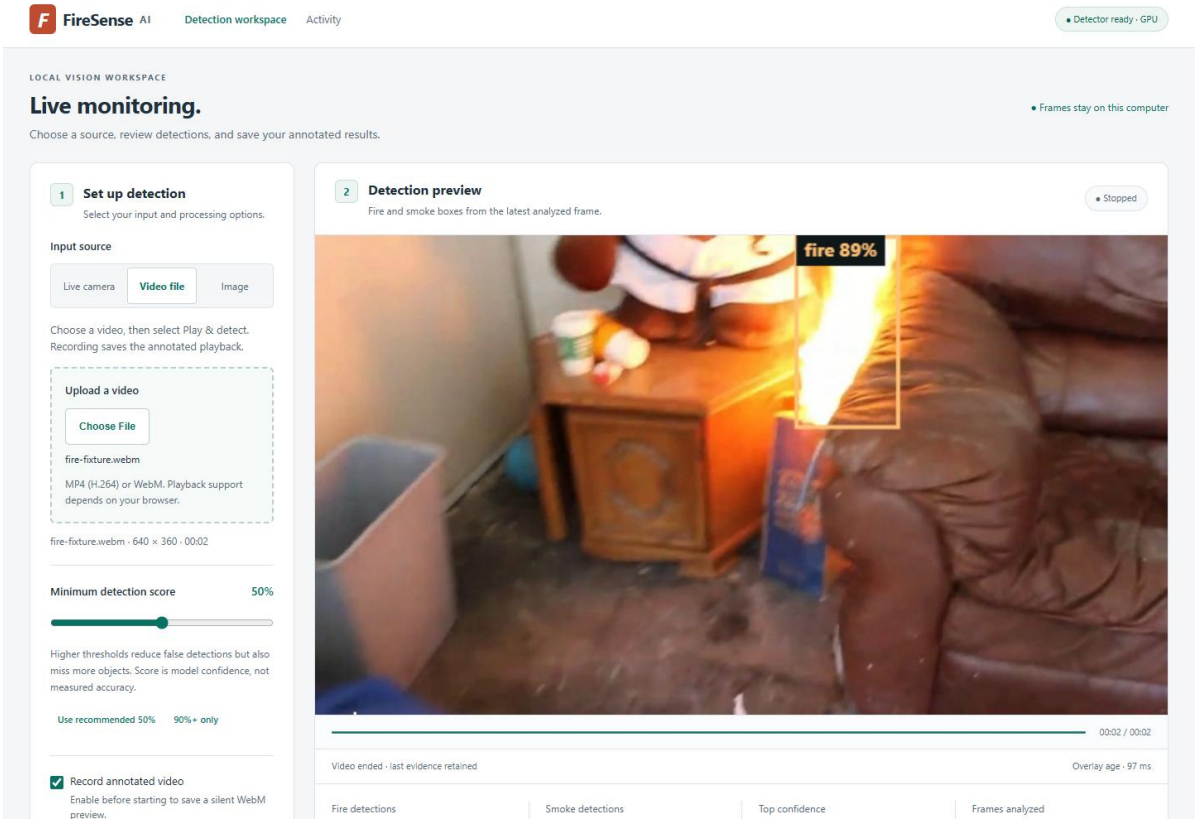


Figure 9 Current local dashboard with source controls, detection preview and informative status. This is an actual browser capture from the completed interface, cropped for readability.

The score filter is presented as a minimum retained score. A low displayed score should prompt examination of the scene and model behaviour. A score slider cannot repair a missed flame or turn an incorrect box into a correct one. For that reason, the current page keeps the raw prediction scores and explains the trade-off when a stricter filter is selected.

6.3 Annotated media downloads

PNG export captures the displayed source frame and its annotations. The browser test compared the resulting image with the unannotated source and found changed pixels from the overlay. The WebM export was decoded and checked for dimensions and playback duration. A three-second fixture produced about 3.065 seconds of recorded media, with playback observed at about 3.304 seconds in the check.



Figure 10 Downloaded annotated PNG from the interface. The exported file contains the detection overlay rather than only the original input image.

The supplied portrait video also produced a playable processed recording. The saved file was approximately 3.38 MB and decoded to 337 frames at 405 by 720 pixels. Container or decoder frame-rate metadata should not be treated as inference throughput. The recording captures a preview at requested timing, while inference samples frames independently.

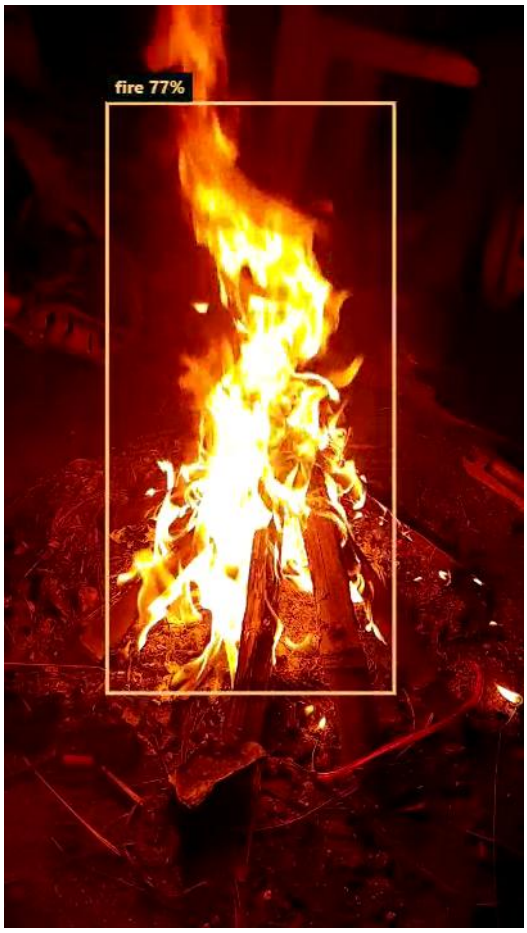


Figure 11 Decoded frame from the processed portrait-video download. The visible fire box and raw score confirm that the annotation is present in the exported media.

6.4 Verification evidence

The interface checks cover invalid inputs, partial recordings, source changes, fallback behaviour and retention of an earlier completed clip. The local service returned successful responses during the reported development-video replays, and no browser page errors were recorded in those replays. These observations establish that the inspected failure was not simply a failed upload or an empty server response.

The tests do not establish performance on all browsers, codecs or camera devices. Browser media support varies, and recording is bounded to limit local memory use. The interface exposes errors instead of silently reporting a successful export when recording cannot be completed. Manual review remains necessary when a downloaded clip is intended for assessment or presentation.

7 RESULTS AND DISCUSSION

7.1 Current held-out test results

The final benchmark uses 1,295 usable test images containing 892 fire boxes and 689 smoke boxes. The selected domain-adapted nano model was evaluated at 416 pixels. At a score filter of 0.50 with class-aware matching at IoU 0.50, it produced 1,083 true positives, 81 false positives and 498 false negatives. The AP figures are computed across score operating points, so they should not be interpreted as the accuracy of a single chosen threshold.

Table 9 Current held-out test results

Class	Objects	Precision at 0.50	Recall at 0.50	AP at IoU 0.50	AP at IoU 0.50 to 0.95
Fire	892	93.36%	67.83%	83.33%	48.23%
Smoke	689	92.64%	69.38%	84.15%	48.46%
Overall	1581	93.04%	68.50%	83.74%	48.35%

Overall precision and recall use summed object counts. Overall AP is the mean of the two class AP values. The per-class fixed-threshold counts are 605 true positives, 43 false positives and 287 false negatives for fire, and 478 true positives, 38 false positives and 211 false negatives for smoke. Those denominators matter: a high fraction of retained boxes is correct, but roughly one third of annotated objects is still missed at the default threshold.

7.2 Precision recall curve

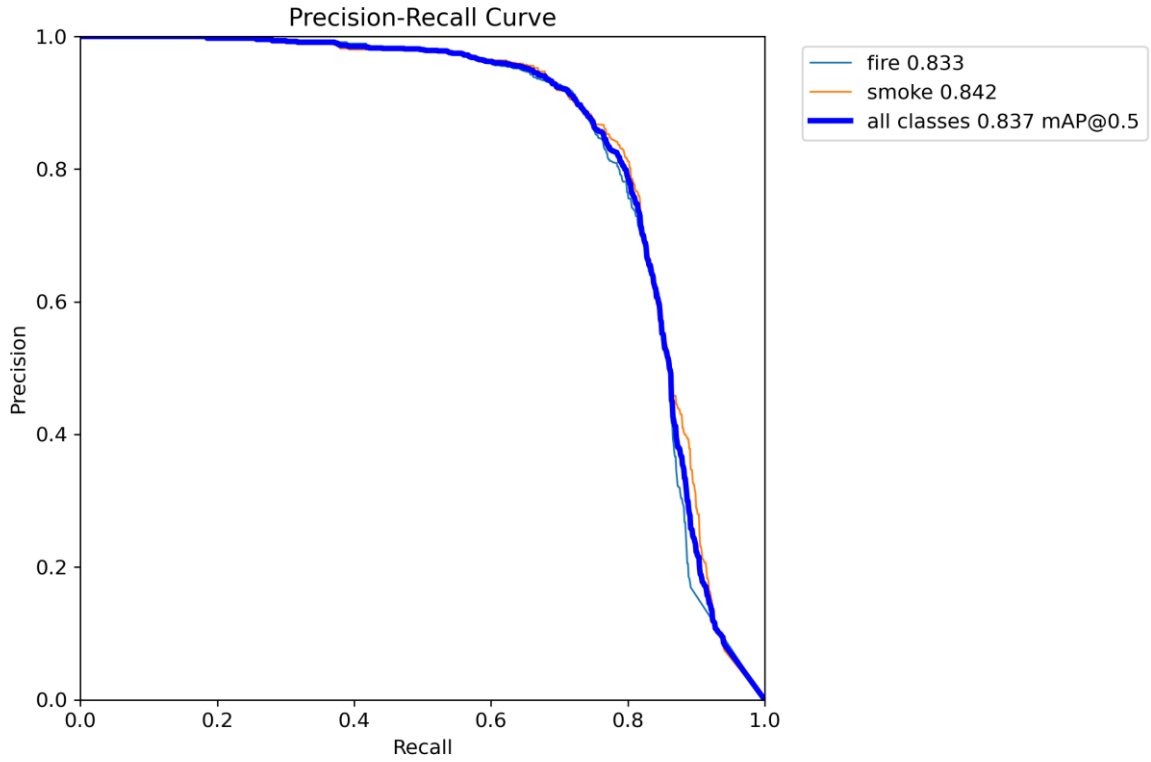


Figure 12 Native precision-recall curves for the selected model on the test split. Fire AP at IoU 0.50 is 0.833 and smoke AP is 0.842, giving mean AP of about 0.837. Source saved domain test run.

The curve describes the effect of retaining progressively more detections as the score threshold changes. It allows the reader to see why one score cutoff cannot maximize precision and recall together. A detection application that prioritizes finding more possible fire regions may accept more false positives at a lower threshold. The current default of 0.50 gives relatively precise retained boxes, but the remaining false negatives prevent a claim of dependable early warning.

7.3 Confusion matrix

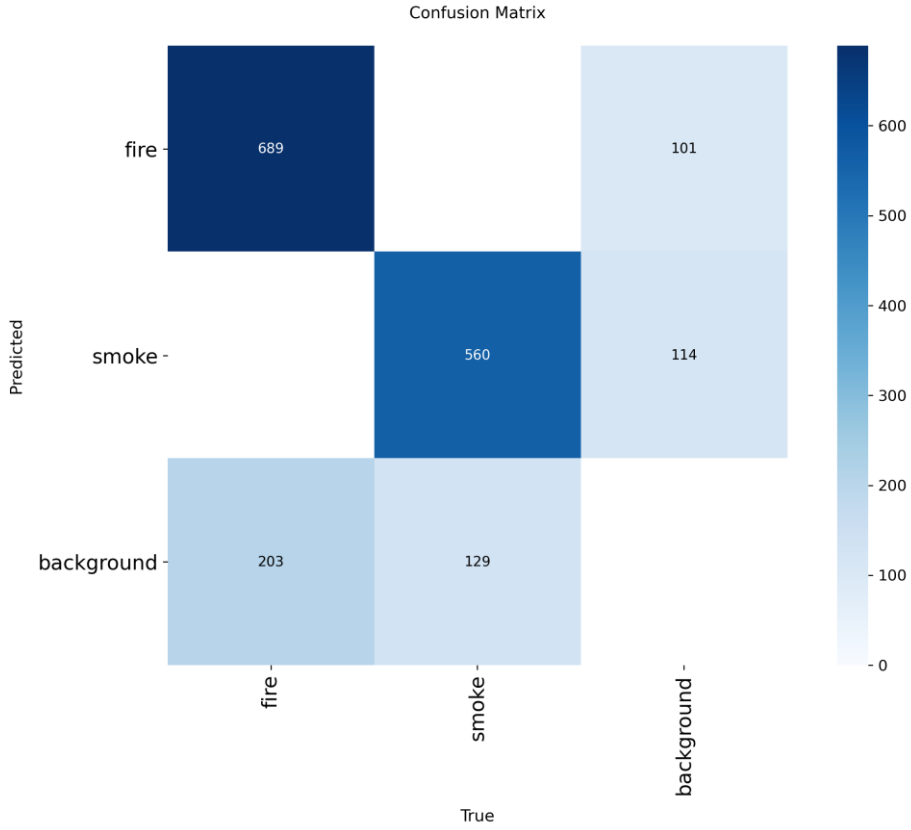


Figure 13 Native Ultralytics confusion matrix for the current test run. Its effective score threshold is 0.25 and its box-matching IoU is 0.45. These defaults differ from the fixed-threshold tables, which use score 0.50 and match IoU 0.50.

In the native detection matrix, rows represent predicted classes and columns represent true classes. The background row captures missed annotated objects, while the background column captures predictions without an accepted match. The figure includes these background outcomes because ordinary class accuracy would hide both missed objects and extra boxes. Its counts are not interchangeable with the project's stricter fixed-threshold table.

This difference in evaluation settings explains why a reader may see different totals in a native plot and a summary table. The report retains the native figure as produced and states its settings rather than editing its numbers to make the presentations appear identical. The saved prediction file allows a consistent alternative threshold analysis without running the model again.

7.4 Confidence threshold analysis

Table 10 Measured confidence operating points

Minimum score	Precision	Recall	True positives	False positives	False negatives
0.25	83.54%	77.36%	1223	241	358
0.50	93.04%	68.50%	1083	81	498
0.75	98.61%	40.35%	638	9	943
0.90	100.00%	4.68%	74	0	1507

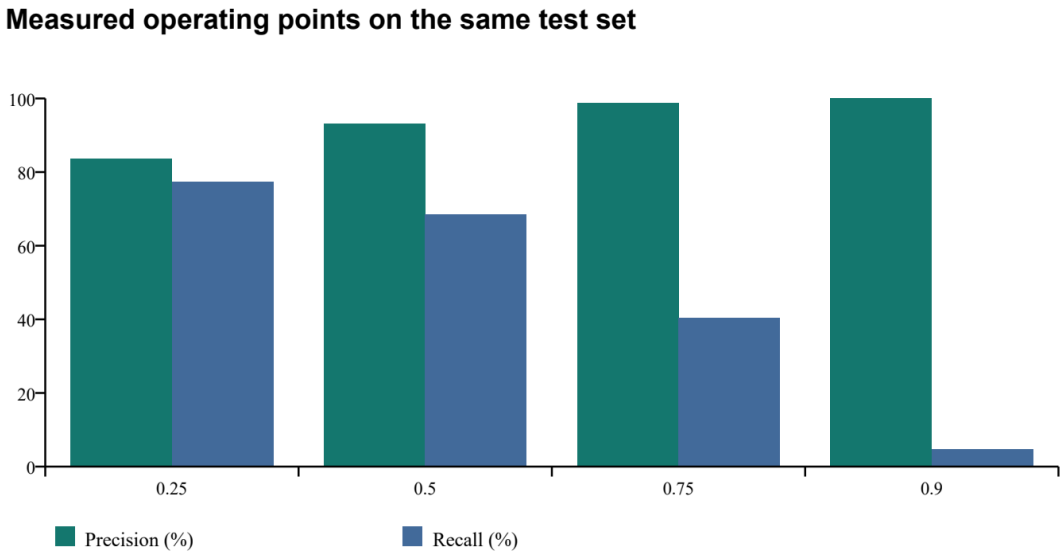


Figure 14 Precision and recall at four measured score filters on the same test set. These are discrete operating points, not a confidence-calibration curve.

At 0.90, the retained detections had no false positives in this test evaluation, but only 74 of 1,581 annotated objects were matched. The filter missed 95.32% of objects. This small retained sample cannot justify a claim of perfect precision in future scenes, and the score is not a calibrated probability that a box is correct. The request for consistently higher scores is therefore better addressed by training coverage and error review than by changing the displayed number.

7.5 Model comparison

Table 11 Model comparison with completed training budgets

Checkpoint	Completed training	Parameters	Test mAP at 0.50	Test mAP at 0.50 to 0.95
YOLOv8 small	8 epochs	11.13 million	73.50%	39.17%
YOLOv8 nano original	20 epochs	3.01 million	82.02%	47.22%
YOLOv8 nano refined	20+4 epochs	3.01 million	82.62%	48.53%
YOLOv8 nano domain adapted	20+4+3 epochs	3.01 million	83.74%	48.35%

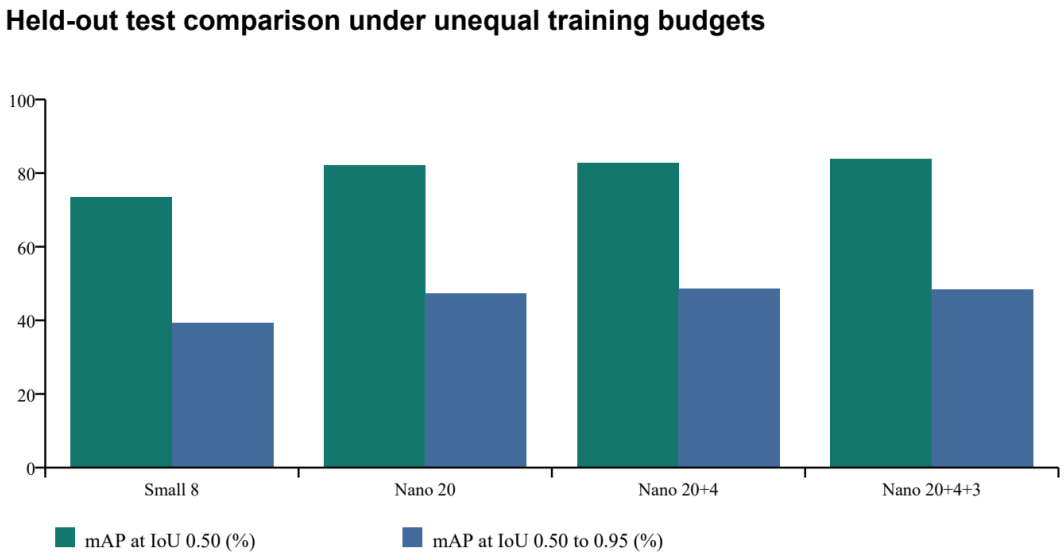


Figure 15 Model comparison on the retained test split at 416 pixels. Training budgets differ. The small model stopped after eight epochs, so this figure does not establish that nano would win under equal training conditions.

The table uses the original baseline evaluation for the original nano and small checkpoints. A later nano re-evaluation at batch four yielded 82.03% mAP@0.5 rather than 82.02%; the difference is a small run-level numerical variation. It does not change the recommendation. All reported comparisons use the same prepared test split, class definitions and input size.

Compared with the refined checkpoint, domain adaptation raised test mAP@0.5 by about 1.12 percentage points and lowered mAP@0.5:0.95 by about 0.18 points. Default-threshold precision rose from 92.15% to 93.04%, while recall rose from 68.31% to 68.50%. The change improves the observed development case and broad-overlap AP, but does not improve every

localization metric. The refined checkpoint remains a useful reference when precise box overlap is the main criterion.

The current recommendation is the domain-adapted nano for this local prototype because it recovers the known close-up failure while retaining validation localization quality and a small model footprint. A properly controlled future comparison should train small and nano for comparable budgets, repeat runs across seeds and evaluate on independently sourced videos before making a stronger model-size recommendation.

7.6 Training and validation observations

Table 12 Adaptation epoch log

Adaptation epoch	Cumulative seconds	Training box loss	Training class loss	Validation mAP at 0.50	Validation mAP at 0.50 to 0.95
1	173.18	1.1953	0.7826	88.44%	54.58%
2	333.11	1.1039	0.6798	90.04%	55.58%
3	480.75	1.0633	0.6375	90.75%	56.98%

The epoch log shows lower training losses and higher validation AP during the short adaptation stage. A separate final validation call reports 90.77% mAP@0.5 and 57.01% mAP@0.5:0.95. The refined checkpoint's validation mAP@0.5:0.95 was 57.00%, so the final candidate met the acceptance floor with essentially unchanged validation localization quality. The final validation call and the epoch log are separate evaluations and retain their own rounded values.

Validation AP exceeds test AP for the current model. Differences in scene content, object sizes or annotation difficulty could contribute, but no controlled analysis here isolates the cause. It would be premature to attribute the whole gap to overfitting or to a specific source. The practical implication is to use the held-out test values when describing benchmark performance rather than quoting the higher validation number as the final result.

7.7 Correct detections and failure examples



Figure 16 Three successful current-model test examples at score 0.50. Left panels show dataset annotations; right panels show model predictions. Green denotes fire and blue denotes smoke. Examples are spaced through the list of images with no counted object errors.

These examples show cases in which class and box overlap satisfy the stated matching rule. They are selected illustrations and cannot stand in for the full test results. A visually reasonable boundary may differ from the annotation, particularly for diffuse smoke. The quantitative tables account for all test images rather than just the examples displayed here.

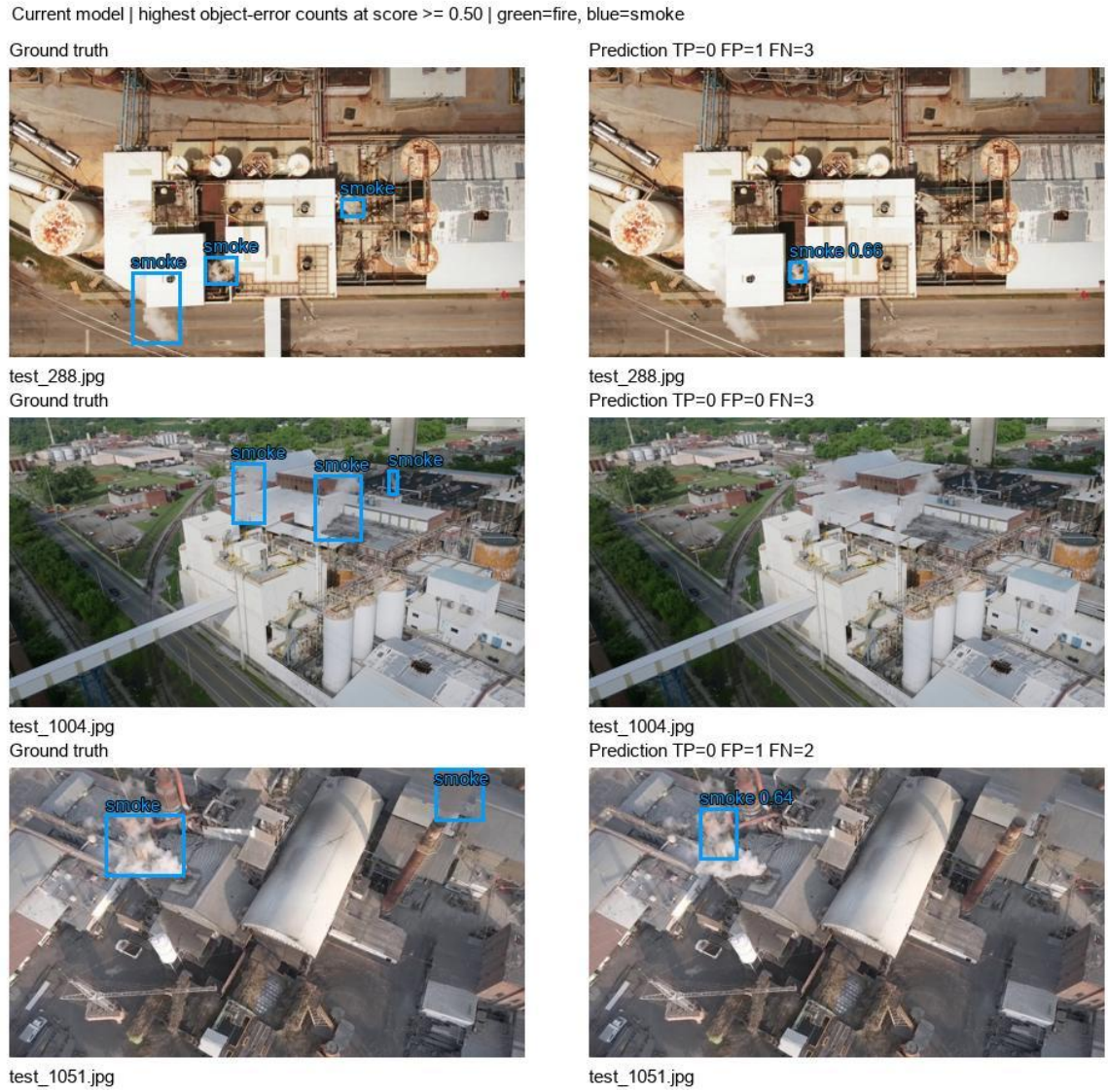


Figure 17 Current-model test images with the highest combined false-positive and false-negative object counts at score 0.50 and matching IoU 0.50. Ground truth is shown alongside predictions so missing and extra boxes can be inspected.

The failure gallery is selected by a recorded rule: descending total object errors. It exposes difficult or incompletely matched cases rather than searching for a flattering screenshot. A false negative may result from a low score, absent candidate box, wrong class or insufficient overlap. A false positive may be a spurious region or a duplicate box without an unused match. Those categories need direct inspection before choosing the next training change.

The displayed failures are industrial scenes with smoke annotations. In test_1004.jpg, all three annotated smoke regions are missed at the default threshold. In test_288.jpg and test_1051.jpg, a retained smoke box is visible but fails the required overlap with an annotation. These cases help explain the gap between broad presence detection and stricter localization AP. They also show why a displayed box alone is not enough to judge an object as correctly detected.

7.8 Development video investigation

The supplied file, 6401825-hd_1080_1920_30fps.mp4, is a 22.43-second portrait video with 673 frames at 30 frames per second and dimensions of 1080 by 1920 pixels. It contains close-up flames under strong red lighting. Eight fixed frames at indices 0, 96, 192, 288, 384, 480, 576 and 672 were retained for a directly matched comparison.

The earlier refined model retained a fire box in two of those eight frames at score 0.50. After adaptation it retained fire in all eight, with raw scores of about 0.70 to 0.80. The boxes were visually checked against the flame regions. No frame-by-frame ground-truth boxes were created for this clip, so the result measures positive-frame coverage in a known case, not object-detection AP or a general video accuracy rate.

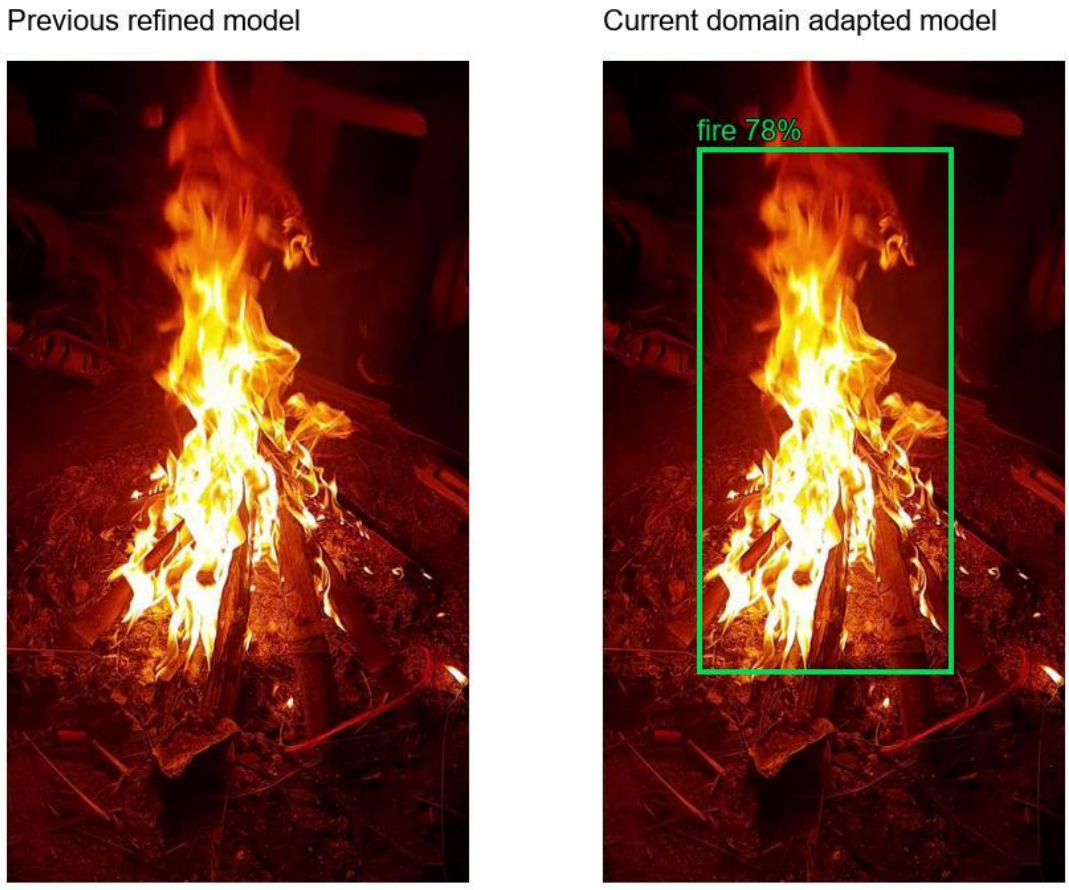


Figure 18 Same development frame at 9.6 seconds before and after adaptation, using a 0.50 score filter. The previous refined model has no retained box here; the current model retains a fire box. Both panels use the identical source image.

Table 13 Development-video observations

Development check	Earlier refined model	Current domain-adapted model
Same eight fixed frames with retained fire	2 of 8	8 of 8
Full browser replay before export redesign	21 of 130 analysed frames	175 of 175 analysed frames
Later replay with processed-video download	Not repeated in that configuration	151 of 151 analysed frames

Browser replay counts differ because requests sample the next available decoded frame after inference completes. Concurrent activity and recording can change the number of analysed frames. The fixed-frame row is the appropriate matched comparison. Successful responses and no recorded page errors during the replays support the diagnosis that the earlier failure came from detection coverage rather than a broken upload path.

The video was excluded from training, but it influenced diagnosis and candidate selection. Its result must therefore remain development evidence. A positive-only fire clip cannot measure false alarms on red lighting without fire, nor prove performance on small distant flames. Those are priorities for the next independent evaluation.

7.9 What the results mean for the project

The model and interface now provide a usable local demonstration with auditable evidence. The selected model usually gives correct retained boxes on this test distribution at the default threshold, while its recall leaves a substantial number of annotated objects undetected. The lower AP under stricter IoU thresholds also shows that box boundaries remain less reliable than broad presence localization.

The close-up recovery is useful because it addresses an observed user failure without rescaling scores. It remains a targeted improvement. The result should be presented with the benchmark tables, error examples and development-data disclosure so that a reader can judge its scope. Neither a successful demonstration clip nor a high confidence score is sufficient evidence for unattended safety use.

8 LIMITATIONS

8.1 Dataset and evaluation limits

The split follows the dataset release, with exact duplicates removed. It has not been proven disjoint by original recording, location or camera. Related scenes may appear across splits. A more demanding evaluation would group images by source sequence and hold out entire sources. Until such identifiers are recovered, the benchmark describes performance on the retained dataset split rather than a verified source-independent deployment distribution.

Background-only scenes are scarce, especially in the test split. The model needs additional evaluation on sunlight, reflections, orange clothing, displays, vehicle lights, steam, fog and dust. These are plausible confusion conditions to test, not measured failure rates from this project. The current two-class task also does not distinguish the cause or severity of a fire, estimate temperature or determine whether smoke is hazardous.

Annotation uncertainty affects localization metrics. Fire and smoke can have irregular, changing boundaries, while bounding boxes summarize those boundaries with rectangles. The experiment uses the supplied annotations and does not include a new inter-annotator agreement study. A manual audit of high-error images would help distinguish model errors from ambiguous or incomplete labels.

8.2 Model comparison limits

The small baseline received fewer completed epochs than nano. Only one recorded training path for each main configuration is reported, so run-to-run variance and confidence intervals are unavailable. The short adaptation was selected against a known development video. It may improve that appearance while leaving other lighting conditions unchanged or weaker. The small fall in test mAP@0.5:0.95 relative to refinement should remain visible when presenting the recommendation.

The confidence output has not been calibrated against a separate calibration set. A displayed value such as 0.80 should not be described as an 80% probability of a correct box. The measured threshold table is more informative for the current dataset. Further work could assess calibration and operating costs using independent data, but it should preserve a separate final test set.

8.3 Runtime and usability limits

The browser samples frames rather than analysing every video frame. Rapid events between sampled frames may be missed, and an overlay can lag behind a moving object. Stale-box expiry reduces visible delay but cannot reconstruct a missed event. No standardized camera-to-alert latency or sustained multi-camera throughput has been measured.

Video export records the rendered preview without audio. It is bounded by browser support, local memory and duration limits. The exported video is not an evidentiary copy of every original frame. Tests used actual uploaded media and a synthetic camera fixture, but a physical webcam was not part of the recorded verification. Automated accessibility scans also do not replace direct testing with assistive-technology users.

8.4 Safety and deployment limits

The application is a research prototype. It has no physical suppression controller, certified alarm path, installation assessment or redundancy mechanism. It should not be used to decide that an occupied space is safe. A deployment beyond local academic review would need a broader hazard assessment, approved response procedures, monitoring, data governance and sustained field validation.

The model and dataset provenance also constrain reuse. Noncommercial dataset terms and relevant software licenses must be reviewed before distribution or commercial use. The project does not claim that saving a trained checkpoint resolves those obligations. Signature and approval fields in this dissertation remain for the institution and named participants to complete.

9 CONCLUSION

9.1 Completed outcome

FireSense AI now provides an end-to-end fire and smoke detection project with public data preparation, trained YOLOv8 checkpoints, held-out evaluation and a local browser interface. The current domain-adapted nano model achieves 83.74% mAP@0.5 and 48.35% mAP@0.5:0.95 on 1,295 test images. At score 0.50, precision is 93.04% and recall is 68.50%. These values describe useful detection capability and a material missed-object limit.

The project also records a concrete generalization failure and its targeted correction. Training-only close-up and red-light variations improved retained fire coverage from two to eight of eight fixed development frames. The selected checkpoint retained validation localization performance, while test localization AP fell slightly compared with the earlier refinement. The report keeps that trade-off explicit.

The interface makes the result accessible through camera, video and image sources and supports annotated PNG and silent WebM downloads. Saved evidence connects the visible application to its model identity and evaluation files. The work meets the practical objective of a runnable academic prototype, with broader deployment claims left for further validation.

9.2 Recommended next work

The next evaluation should collect source-independent videos that include small distant flames, nighttime scenes, difficult smoke and long non-fire sequences. Those recordings should remain untouched during training and selection. Box annotations and frame-level event labels would allow separate measurement of object AP, false alarms per unit time and time to first retained detection.

A fair model comparison should give nano and small comparable training budgets and repeat the experiment across seeds. Error analysis should then guide additional training data rather than simply increasing epochs. If the model is intended for early warning, a threshold should be chosen using the cost of missed events and false alarms, supported by an independent validation set.

10 REFERENCES

[1] J. Redmon, S. Divvala, R. Girshick and A. Farhadi. You Only Look Once Unified Real-Time Object Detection. CVPR, 2016. https://arxiv.org/abs/1506.02640

[2] Ultralytics. YOLOv8 model documentation. https://docs.ultralytics.com/models/yolov8/ Accessed 26 October 2026.

[3] B. Peng and T.-K. Kim. Home Fire Dataset version 1.0.0. Official dataset repository. https://github.com/PengBo0/Home-fire-dataset Accessed 26 October 2026.

[4] B. Peng and T.-K. Kim. YOLO-HF Early Detection of Home Fires Using YOLO. IEEE Access, volume 13, pages 79451 to 79466, 2025. https://doi.org/10.1109/ACCESS.2025.3566907

[5] T.-Y. Lin et al. Microsoft COCO Common Objects in Context. ECCV, 2014. https://arxiv.org/abs/1405.0312

[6] Creative Commons. Attribution-NonCommercial 4.0 International license. https://creativecommons.org/licenses/by-nc/4.0/ Accessed 26 October 2026.

[7] Ultralytics. YOLO performance metrics. https://docs.ultralytics.com/guides/yolo-performance-metrics/ Accessed 26 October 2026.

[8] Mozilla contributors. MediaRecorder and HTMLCanvasElement captureStream documentation. https://developer.mozilla.org/en-US/docs/Web/API/MediaRecorder and https://developer.mozilla.org/en-US/docs/Web/API/HTMLCanvasElement/captureStream Accessed 26 October 2026.

[9] FireSense AI local project evidence. Data audit, model selection profiles, training CSV logs, saved prediction JSON, browser download checks and accessibility records. Paths and provenance are listed in Appendix B. Recorded project snapshot 26 October 2026.

APPENDIX A RUN INSTRUCTIONS

A.1 Environment and application

Run the following commands from D:/fire ai in PowerShell. The trained weights and prepared dataset are already present in the completed local project. On a fresh machine, create the virtual environment, install the pinned requirements and obtain the dataset under its license before running preparation. GPU-specific package installation may need the matching PyTorch CUDA wheel source.

```
Set-Location 'D:/fire ai'
.venv/Scripts/python.exe -m pip install -r requirements.txt
.venv/Scripts/python.exe dashboard.py
```

Open http://localhost:8765/ in a supported browser. Select an image, video or camera source. Keep the default 0.50 score filter for the documented deployment setting. Wait for model status and a successful detection response before judging the output. Stop a video or allow it to end before downloading its completed WebM recording. A frame can be saved as PNG from the download controls.

```
.venv/Scripts/python.exe dashboard.py --device cpu --port 8766
```

The alternative command selects CPU inference and a different local port. CPU response time has not been benchmarked in this dissertation. Camera access requires browser permission. A refused permission or unsupported codec should be resolved as an input issue before interpreting an empty overlay as a model result.

A.2 Preparation training and inference

```
.venv/Scripts/python.exe pipeline.py prepare
.venv/Scripts/python.exe pipeline.py train --size n --epochs 20
.venv/Scripts/python.exe pipeline.py train --size s --epochs 20
.venv/Scripts/python.exe pipeline.py predict --source 'path/to/image-or-video'
```

The small command requests 20 epochs; the recorded historical small run completed eight. A new 20-epoch result would be a new experiment and must not be substituted silently into the comparison. Prediction uses the active profile unless weights are explicitly overridden. Use --conf 0.25 to inspect a lower score filter or --weights with a saved model path to compare checkpoints.

```
.venv/Scripts/python.exe scripts/adapt_detector.py
.venv/Scripts/python.exe scripts/adapt_detector.py --train
```

The first command prepares the training-derived views. The second trains a candidate. Reruns protect the deployed checkpoint by using a distinct candidate path; inspect the printed output and validate that candidate before changing the active profile. The saved historical adaptation settings and augmentation manifest remain the source of truth for the result documented here.

A.3 Evaluation and checks

```
.venv/Scripts/python.exe evaluate.py --split test
```

```
.venv/Scripts/python.exe scripts/optimize_detector.py --weights models/yolov8n_domain.pt --sizes 416 --split test
```

```
.venv/Scripts/python.exe scripts/optimize_detector.py --weights models/yolov8n_domain.pt --sizes 416 --split val
```

The first command evaluates the baseline nano and small models. The second evaluates the current domain checkpoint, writes native curves and computes fixed-threshold metrics. Keep validation for selection and preserve test results as a report of the chosen model. Do not repeatedly tune against the final test set and then describe it as independent selection evidence.

The current dashboard, export and augmentation tests live under the project tests and scripts directories. Browser checks require their recorded browser tooling and an active local server. Their saved JSON evidence is included so that the documented outcome can be inspected even when those browser tools are unavailable on another machine.

APPENDIX B EVIDENCE AND PROVENANCE

B.1 Dataset and training records

Table 14 Dataset and training evidence

Evidence path	Supports
evidence/data_audit.json	Usable split counts and excluded image reasons
evidence/manifest.json	Prepared image manifest
evidence/duplicate_annotation_check.json	Duplicate removal and annotation differences
runs/train/yolov8n/results.csv	Original nano 20-epoch training trends
runs/train/yolov8s/results.csv	Eight completed small-model epochs
runs/optimization/nano_refinement2/	Four-epoch refinement arguments and logs
runs/optimization/domain_adaptation2/	Three-epoch adaptation arguments and logs
evidence/user_video/augmentation_manifest.json	Sources and transforms for 600 derived views

B.2 Metrics and model identity

Table 15 Metrics and checkpoint evidence

Evidence path	Supports
evidence/test_metrics.json	Original nano and small test comparison
evidence/optimization_test_yolov8n_refined.json	Refined checkpoint test values
evidence/optimization_test_yolov8n_domain.json	Current test AP and threshold counts
evidence/optimization_val_yolov8n_domain.json	Current final validation values
evidence/user_video/selection_contract.json	Candidate acceptance criteria
evidence/optimization_selection.json	Active weights, configuration and hash
runs/optimization/test_yolov8n_domain_416/predictions.json	Per-image predictions used for current galleries
reports/thesis_source/figures/gallery_selection.json	Exact examples and matching outcomes shown here

B.3 Development and interface records

Table 16 Development and interface evidence

Evidence path	Supports
evidence/user_video/baseline_raw_frames.json	Earlier model results on eight fixed frames
evidence/user_video/candidate_final.json	Current model results on those same frames
evidence/user_video/source_provenance.json	Video metadata and source hash
evidence/user_video/processed_video_download.json	Later browser replay with export
evidence/download_checks/browser_check.json	Media, source handling and viewport checks
evidence/download_checks/accessibility.json	Automated accessibility scan results
reports/VIDEO_GENERALIZATION_REPORT.md	Detailed diagnosis and selection limitations

The source video's SHA256 is 304197e3a9160ecf2a477c601b6d8e85dae49ffc9a10b5916e7522b5099b3665. It identifies the inspected file, whose original location is C:/Users/Karth/Downloads/6401825-hd_1080_1920_30fps.mp4. The development-video figures are derived from that file. Public release of the clip or dissertation illustrations should confirm permission to redistribute the supplied media; its license was not established by the model experiment.

B.4 Document reproduction

The dissertation source and figure builder are stored in reports/thesis_source/. Figures generated for this document use saved experiment data. Test galleries are reconstructed from current-model prediction JSON and annotations, with an assertion that their full-set counts equal 1083 true positives, 81 false positives and 498 false negatives. Original evaluation plots retain their native labels and settings.

The document follows the supplied MSc Data Science template. The template's stock-market examples were replaced, the VIT logo was retained and the chapter order was preserved. The front-page signature and approval fields are intentionally unfilled. The named student and

guide should review declarations and institutional wording before signing or submitting the document.

APPENDIX C HELD OUT TEST PREDICTION CASES

Each page compares the dataset boxes with predictions from the selected YOLOv8 nano checkpoint. The score filter is 0.50, and a correct match requires the same class and an intersection over union of at least 0.50. Green outlines indicate fire; blue outlines indicate smoke. The first fifteen cases have no counted object errors under this rule. The remaining cases were selected from images with false positives or false negatives. They are examples for review, not a substitute for aggregate test metrics.

Case 01 test_3.jpg

This image contains 1 annotated object(s). The model returned 1 retained box(es), with 1 true positive(s), no false positives and no false negatives at the stated matching settings.

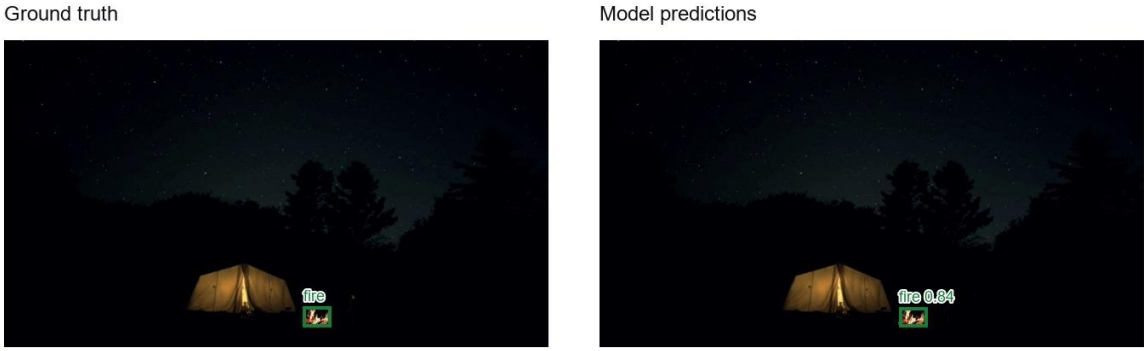


Figure 19 Test case 01, test_3.jpg. Ground truth and current model predictions at score 0.50. Counts use class-aware matching at IoU 0.50.

Case 02 test_95.jpg

This image contains 1 annotated object(s). The model returned 1 retained box(es), with 1 true positive(s), no false positives and no false negatives at the stated matching settings.



Figure 20 Test case 02, test_95.jpg. Ground truth and current model predictions at score 0.50. Counts use class-aware matching at IoU 0.50.

Case 03 test_191.jpg

This image contains 1 annotated object(s). The model returned 1 retained box(es), with 1 true positive(s), no false positives and no false negatives at the stated matching settings.

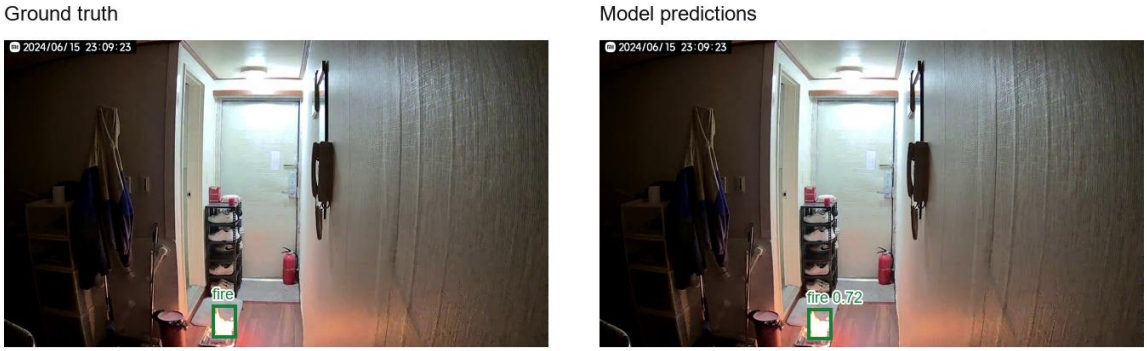


Figure 21 Test case 03, test_191.jpg. Ground truth and current model predictions at score 0.50. Counts use class-aware matching at IoU 0.50.

Case 04 test_295.jpg

This image contains 1 annotated object(s). The model returned 1 retained box(es), with 1 true positive(s), no false positives and no false negatives at the stated matching settings.



Figure 22 Test case 04, test_295.jpg. Ground truth and current model predictions at score 0.50. Counts use class-aware matching at IoU 0.50.

Case 05 test_381.jpg

This image contains 1 annotated object(s). The model returned 1 retained box(es), with 1 true positive(s), no false positives and no false negatives at the stated matching settings.

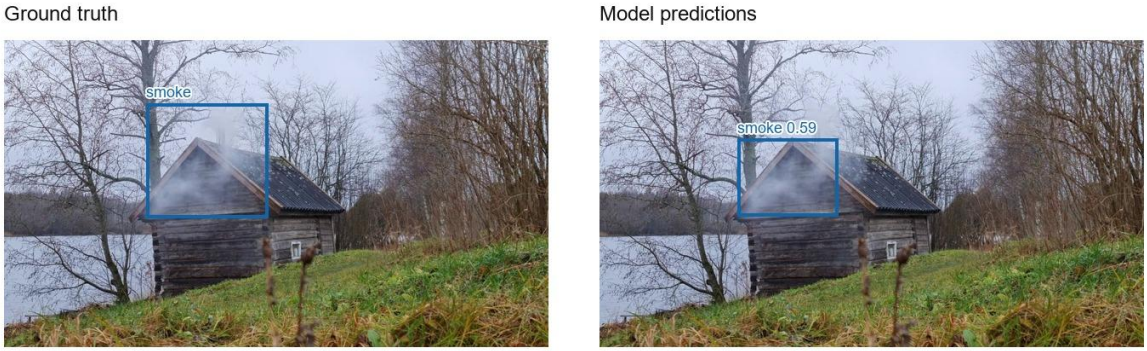


Figure 23 Test case 05, test_381.jpg. Ground truth and current model predictions at score 0.50. Counts use class-aware matching at IoU 0.50.

Case 06 test_481.jpg

This image contains 1 annotated object(s). The model returned 1 retained box(es), with 1 true positive(s), no false positives and no false negatives at the stated matching settings.

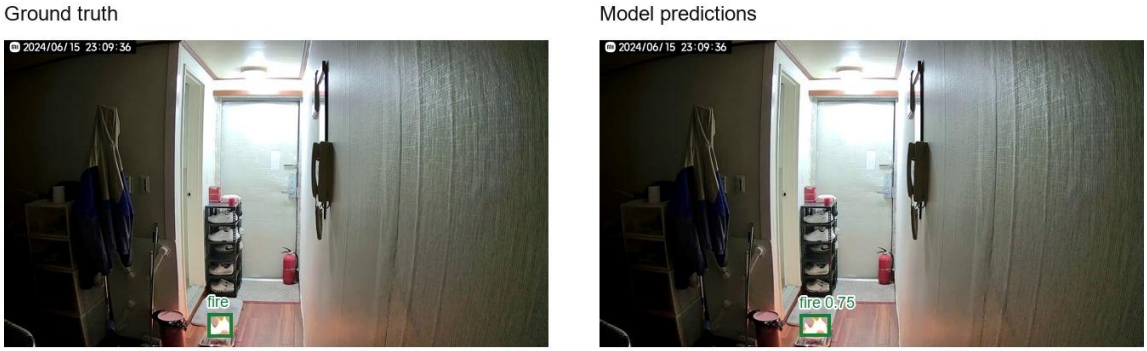


Figure 24 Test case 06, test_481.jpg. Ground truth and current model predictions at score 0.50. Counts use class-aware matching at IoU 0.50.

Case 07 test_572.jpg

This image contains 1 annotated object(s). The model returned 1 retained box(es), with 1 true positive(s), no false positives and no false negatives at the stated matching settings.



Figure 25 Test case 07, test_572.jpg. Ground truth and current model predictions at score 0.50. Counts use class-aware matching at IoU 0.50.

Case 08 test_656.jpg

This image contains 1 annotated object(s). The model returned 1 retained box(es), with 1 true positive(s), no false positives and no false negatives at the stated matching settings.



Figure 26 Test case 08, test_656.jpg. Ground truth and current model predictions at score 0.50. Counts use class-aware matching at IoU 0.50.

Case 09 test_738.jpg

This image contains 1 annotated object(s). The model returned 1 retained box(es), with 1 true positive(s), no false positives and no false negatives at the stated matching settings.



Figure 27 Test case 09, test_738.jpg. Ground truth and current model predictions at score 0.50. Counts use class-aware matching at IoU 0.50.

Case 10 test_835.jpg

This image contains 1 annotated object(s). The model returned 1 retained box(es), with 1 true positive(s), no false positives and no false negatives at the stated matching settings.

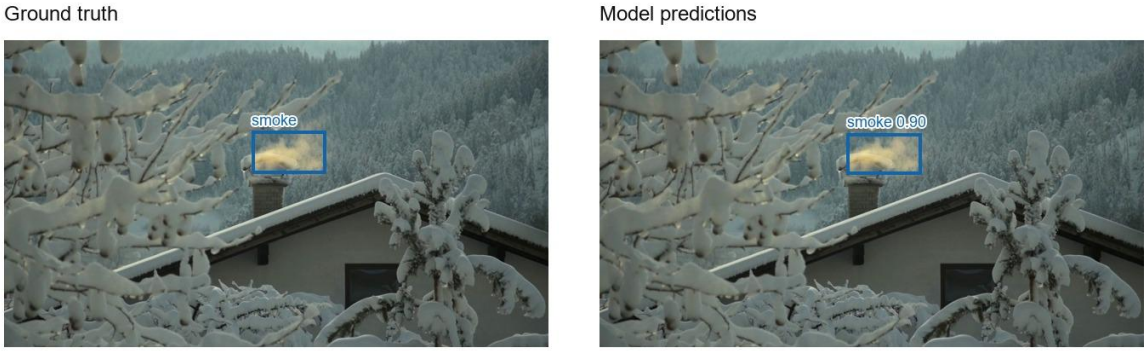


Figure 28 Test case 10, test_835.jpg. Ground truth and current model predictions at score 0.50. Counts use class-aware matching at IoU 0.50.

Case 11 test_923.jpg

This image contains 1 annotated object(s). The model returned 1 retained box(es), with 1 true positive(s), no false positives and no false negatives at the stated matching settings.

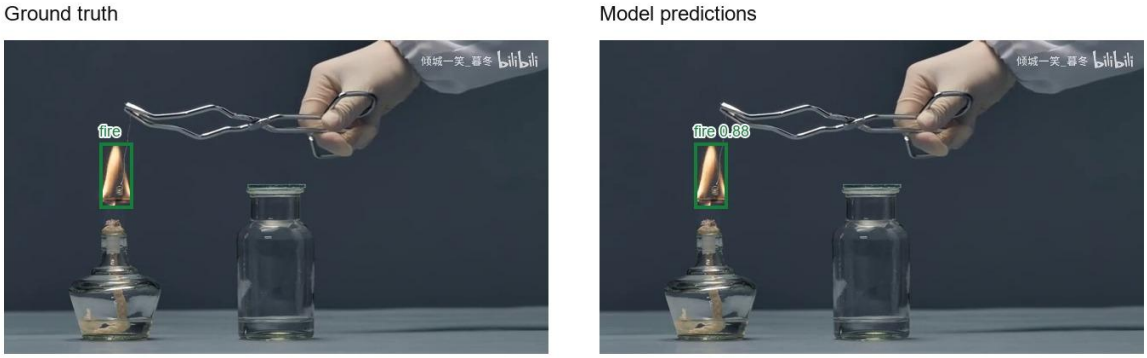


Figure 29 Test case 11, test_923.jpg. Ground truth and current model predictions at score 0.50. Counts use class-aware matching at IoU 0.50.

Case 12 test_1019.jpg

This image contains 1 annotated object(s). The model returned 1 retained box(es), with 1 true positive(s), no false positives and no false negatives at the stated matching settings.



Figure 30 Test case 12, test_1019.jpg. Ground truth and current model predictions at score 0.50. Counts use class-aware matching at IoU 0.50.

Case 13 test_1108.jpg

This image contains 1 annotated object(s). The model returned 1 retained box(es), with 1 true positive(s), no false positives and no false negatives at the stated matching settings.



Figure 31 Test case 13, test_1108.jpg. Ground truth and current model predictions at score 0.50. Counts use class-aware matching at IoU 0.50.

Case 14 test_1209.jpg

This image contains 1 annotated object(s). The model returned 1 retained box(es), with 1 true positive(s), no false positives and no false negatives at the stated matching settings.

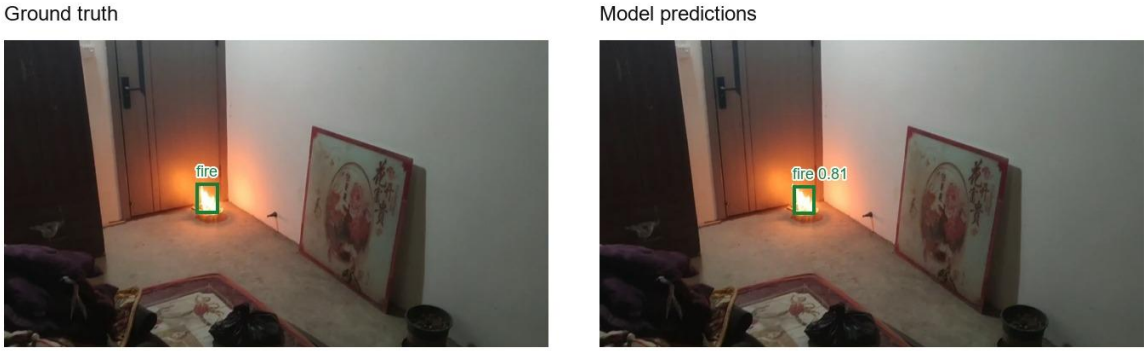


Figure 32 Test case 14, test_1209.jpg. Ground truth and current model predictions at score 0.50. Counts use class-aware matching at IoU 0.50.

Case 15 test_1300.jpg

This image contains 1 annotated object(s). The model returned 1 retained box(es), with 1 true positive(s), no false positives and no false negatives at the stated matching settings.

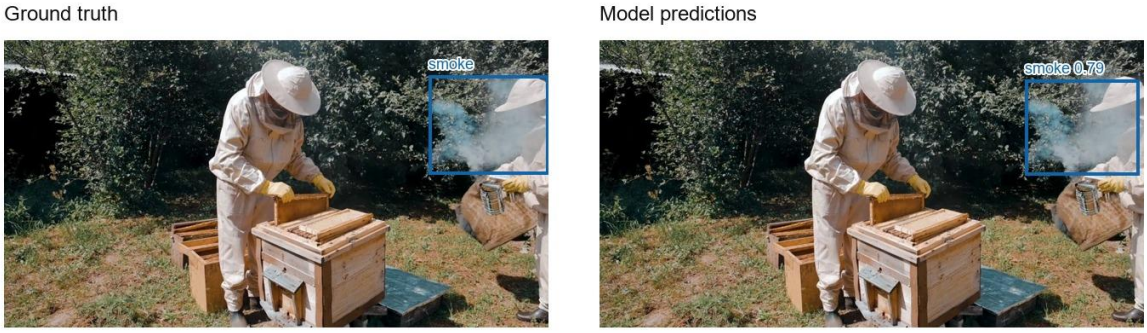


Figure 33 Test case 15, test_1300.jpg. Ground truth and current model predictions at score 0.50. Counts use class-aware matching at IoU 0.50.

Case 16 test_288.jpg

This image contains 3 annotated object(s). The model returned 1 retained box(es), with 0 true positive(s), 1 false positive(s) and 3 false negative(s). A retained box with inadequate overlap counts as an error under the matching rule.



Figure 34 Test case 16, test_288.jpg. Ground truth and current model predictions at score 0.50. Counts use class-aware matching at IoU 0.50.

Case 17 test_255.jpg

This image contains 1 annotated object(s). The model returned 1 retained box(es), with 0 true positive(s), 1 false positive(s) and 1 false negative(s). A retained box with inadequate overlap counts as an error under the matching rule.

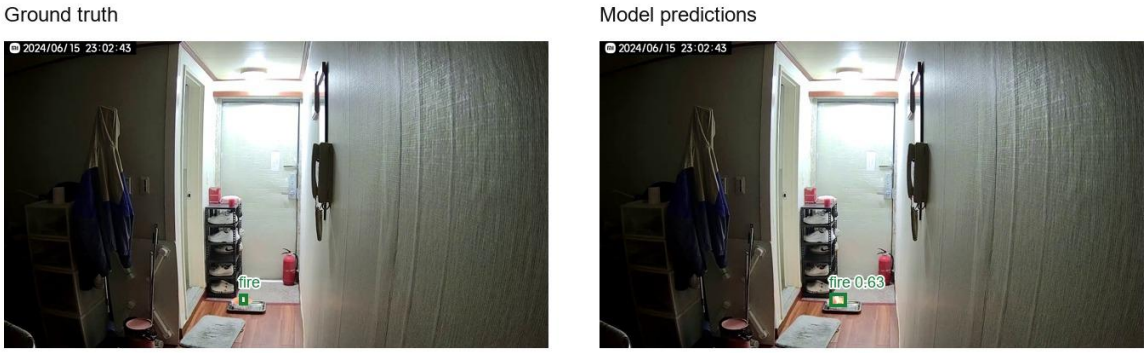


Figure 35 Test case 17, test_255.jpg. Ground truth and current model predictions at score 0.50. Counts use class-aware matching at IoU 0.50.

Case 18 test_908.jpg

This image contains 1 annotated object(s). The model returned 1 retained box(es), with 0 true positive(s), 1 false positive(s) and 1 false negative(s). A retained box with inadequate overlap counts as an error under the matching rule.

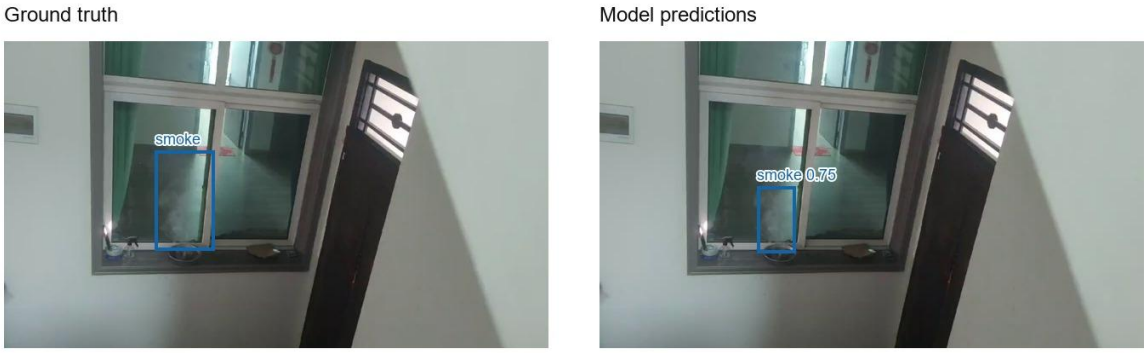


Figure 36 Test case 18, test_908.jpg. Ground truth and current model predictions at score 0.50. Counts use class-aware matching at IoU 0.50.

Case 19 test_15.jpg

This image contains 2 annotated object(s). The model returned 1 retained box(es), with 1 true positive(s), 0 false positive(s) and 1 false negative(s). A retained box with inadequate overlap counts as an error under the matching rule.

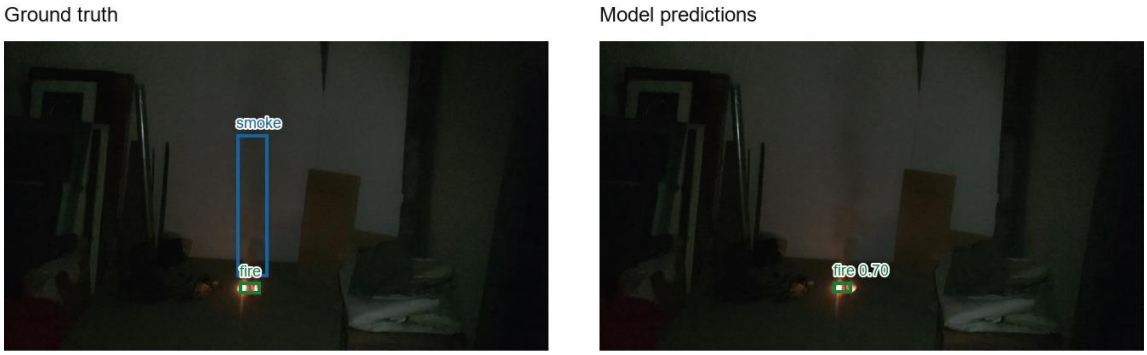


Figure 37 Test case 19, test_15.jpg. Ground truth and current model predictions at score 0.50. Counts use class-aware matching at IoU 0.50.

Case 20 test_126.jpg

This image contains 2 annotated object(s). The model returned 1 retained box(es), with 1 true positive(s), 0 false positive(s) and 1 false negative(s). A retained box with inadequate overlap counts as an error under the matching rule.

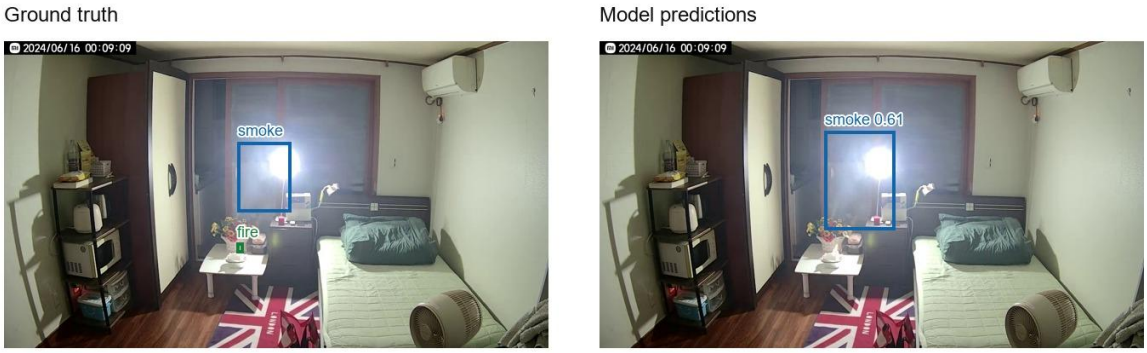


Figure 38 Test case 20, test_126.jpg. Ground truth and current model predictions at score 0.50. Counts use class-aware matching at IoU 0.50.

Case 21 test_221.jpg

This image contains 2 annotated object(s). The model returned 1 retained box(es), with 1 true positive(s), 0 false positive(s) and 1 false negative(s). A retained box with inadequate overlap counts as an error under the matching rule.

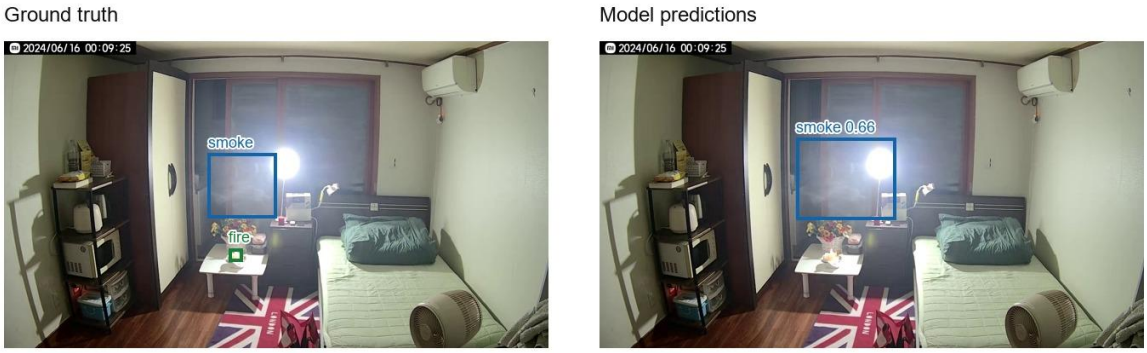


Figure 39 Test case 21, test_221.jpg. Ground truth and current model predictions at score 0.50. Counts use class-aware matching at IoU 0.50.

Case 22 test_327.jpg

This image contains 1 annotated object(s). The model returned 0 retained box(es), with 0 true positive(s), 0 false positive(s) and 1 false negative(s). A retained box with inadequate overlap counts as an error under the matching rule.



Figure 40 Test case 22, test_327.jpg. Ground truth and current model predictions at score 0.50. Counts use class-aware matching at IoU 0.50.

Case 23 test_438.jpg

This image contains 2 annotated object(s). The model returned 1 retained box(es), with 1 true positive(s), 0 false positive(s) and 1 false negative(s). A retained box with inadequate overlap counts as an error under the matching rule.



Figure 41 Test case 23, test_438.jpg. Ground truth and current model predictions at score 0.50. Counts use class-aware matching at IoU 0.50.

Case 24 test_544.jpg

This image contains 1 annotated object(s). The model returned 0 retained box(es), with 0 true positive(s), 0 false positive(s) and 1 false negative(s). A retained box with inadequate overlap counts as an error under the matching rule.



Figure 42 Test case 24, test_544.jpg. Ground truth and current model predictions at score 0.50. Counts use class-aware matching at IoU 0.50.

Case 25 test_687.jpg

This image contains 1 annotated object(s). The model returned 0 retained box(es), with 0 true positive(s), 0 false positive(s) and 1 false negative(s). A retained box with inadequate overlap counts as an error under the matching rule.

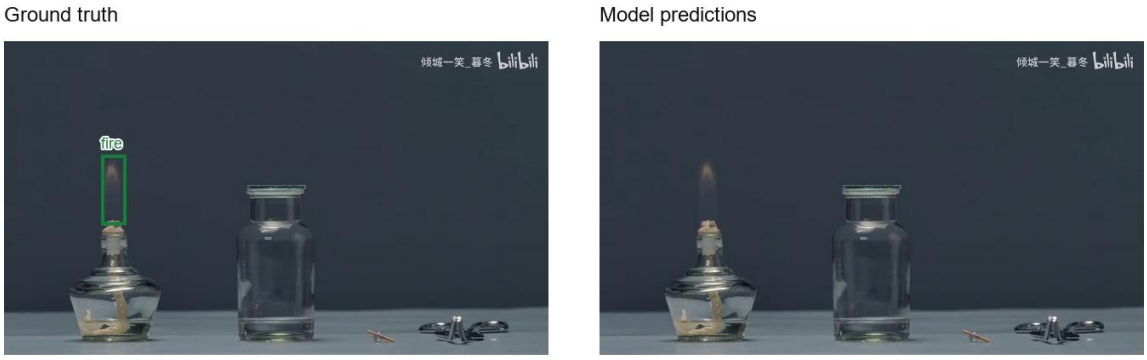


Figure 43 Test case 25, test_687.jpg. Ground truth and current model predictions at score 0.50. Counts use class-aware matching at IoU 0.50.

Case 26 test_794.jpg

This image contains 1 annotated object(s). The model returned 0 retained box(es), with 0 true positive(s), 0 false positive(s) and 1 false negative(s). A retained box with inadequate overlap counts as an error under the matching rule.

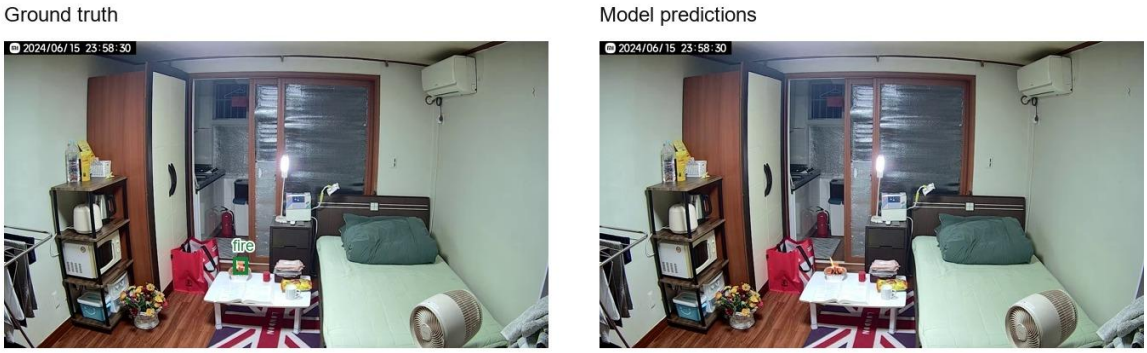


Figure 44 Test case 26, test_794.jpg. Ground truth and current model predictions at score 0.50. Counts use class-aware matching at IoU 0.50.

Case 27 test_940.jpg

This image contains 2 annotated object(s). The model returned 1 retained box(es), with 1 true positive(s), 0 false positive(s) and 1 false negative(s). A retained box with inadequate overlap counts as an error under the matching rule.



Figure 45 Test case 27, test_940.jpg. Ground truth and current model predictions at score 0.50. Counts use class-aware matching at IoU 0.50.

Case 28 test_1082.jpg

This image contains 1 annotated object(s). The model returned 0 retained box(es), with 0 true positive(s), 0 false positive(s) and 1 false negative(s). A retained box with inadequate overlap counts as an error under the matching rule.

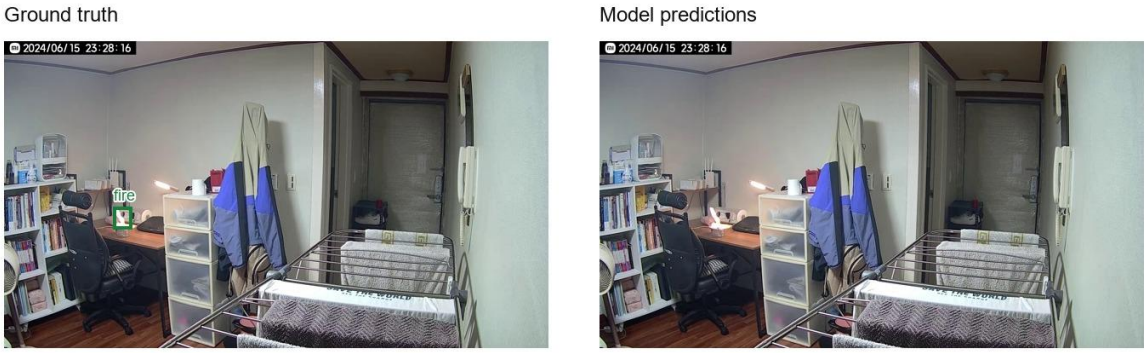


Figure 46 Test case 28, test_1082.jpg. Ground truth and current model predictions at score 0.50. Counts use class-aware matching at IoU 0.50.

Case 29 test_1175.jpg

This image contains 1 annotated object(s). The model returned 0 retained box(es), with 0 true positive(s), 0 false positive(s) and 1 false negative(s). A retained box with inadequate overlap counts as an error under the matching rule.

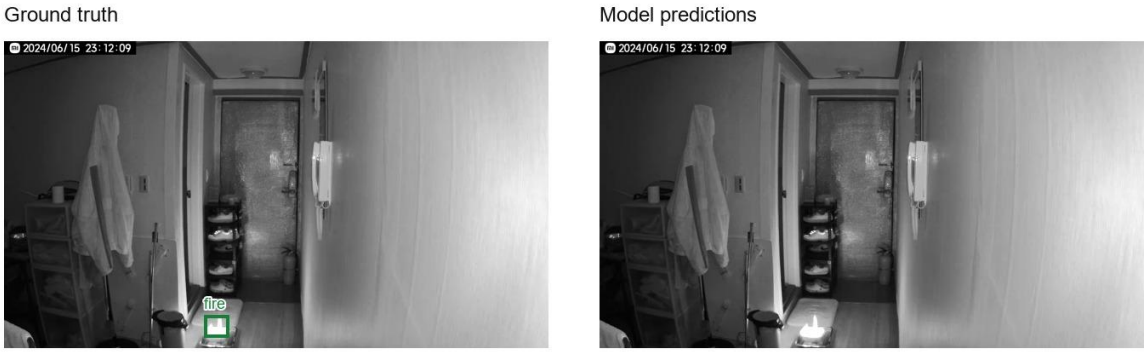


Figure 47 Test case 29, test_1175.jpg. Ground truth and current model predictions at score 0.50. Counts use class-aware matching at IoU 0.50.

Case 30 test_1290.jpg

This image contains 2 annotated object(s). The model returned 1 retained box(es), with 1 true positive(s), 0 false positive(s) and 1 false negative(s). A retained box with inadequate overlap counts as an error under the matching rule.

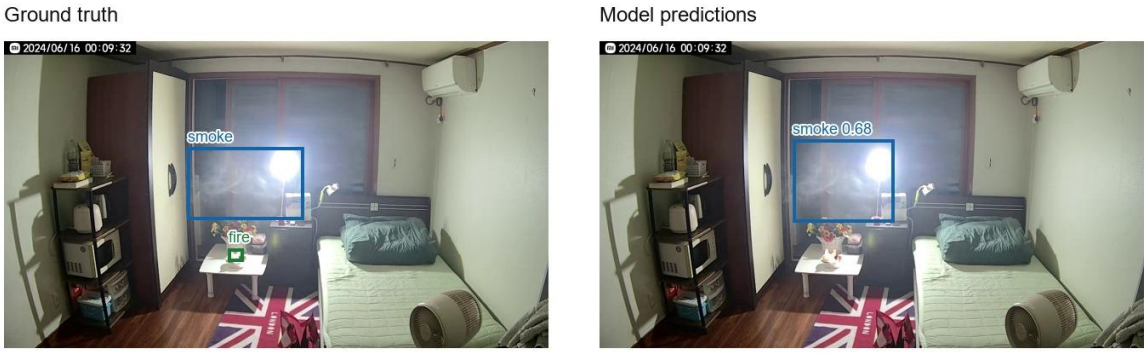


Figure 48 Test case 30, test_1290.jpg. Ground truth and current model predictions at score 0.50. Counts use class-aware matching at IoU 0.50.